CG3201 Machine Learning & Deep Learning

Lecture 2 Support Vector Machine (SVM)
(from Linear Classifiers)

© Copyright National University of Singapore. All Rights Reserved.

Recap Lecture 1

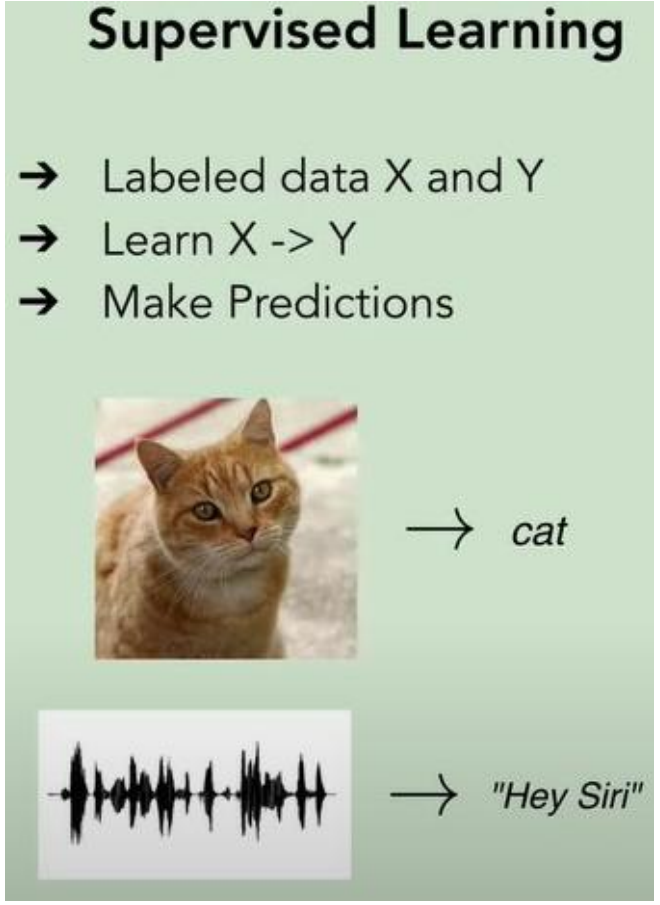
- Traditional Programming vs. Machine Learning vs. Deep Learning.
- Learning problem: **what** kind of *task* or *objective* the learning system is trying to solve.
  - Supervised, unsupervised, reinforcement learning.
  - Semi-supervised learning.
  - Self-supervised learning.
- Learning paradigm: **how** the system *approaches* learning.
  - Instance-based vs. model-based learning.
  - Parametric vs. non-parametric learning.
  - Generative vs. discriminative learning.

© Copyright National University of Singapore. All Rights Reserved.

The Two Categories of Learning Tasks

- Let us just take supervised learning paradigm for ease of understanding.
- Recall, there are two categories of learning tasks:
  - Categorized based on the attributes of the output.

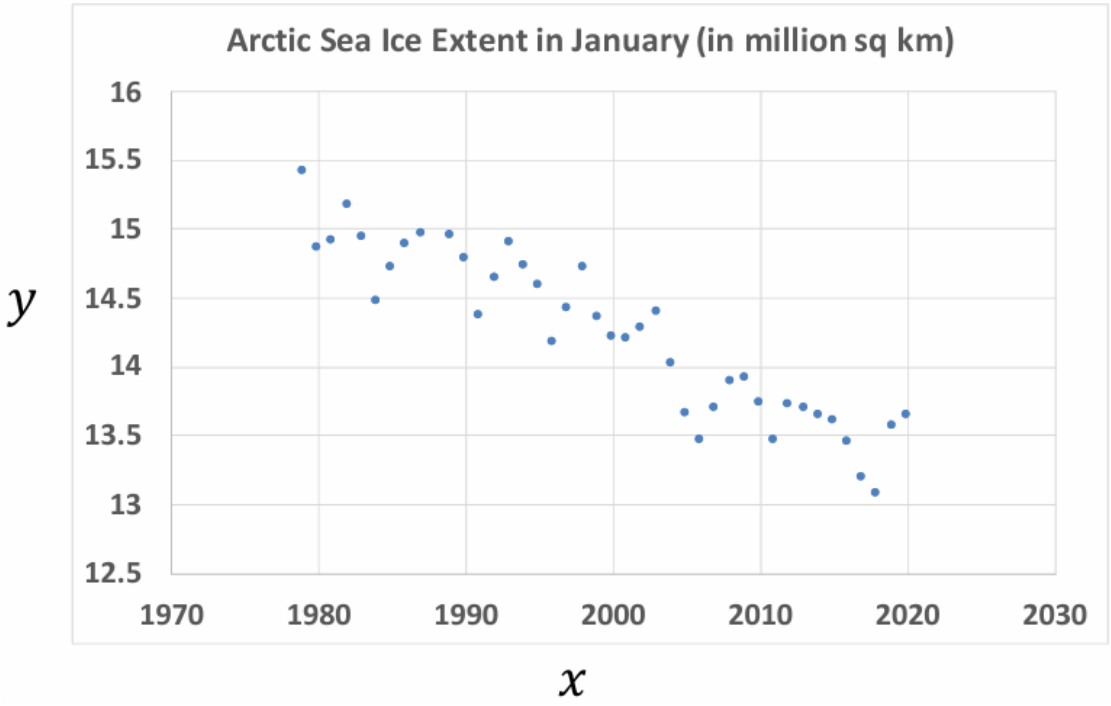
- Quick question: is text generation task a **classification** task or a **regression** task?

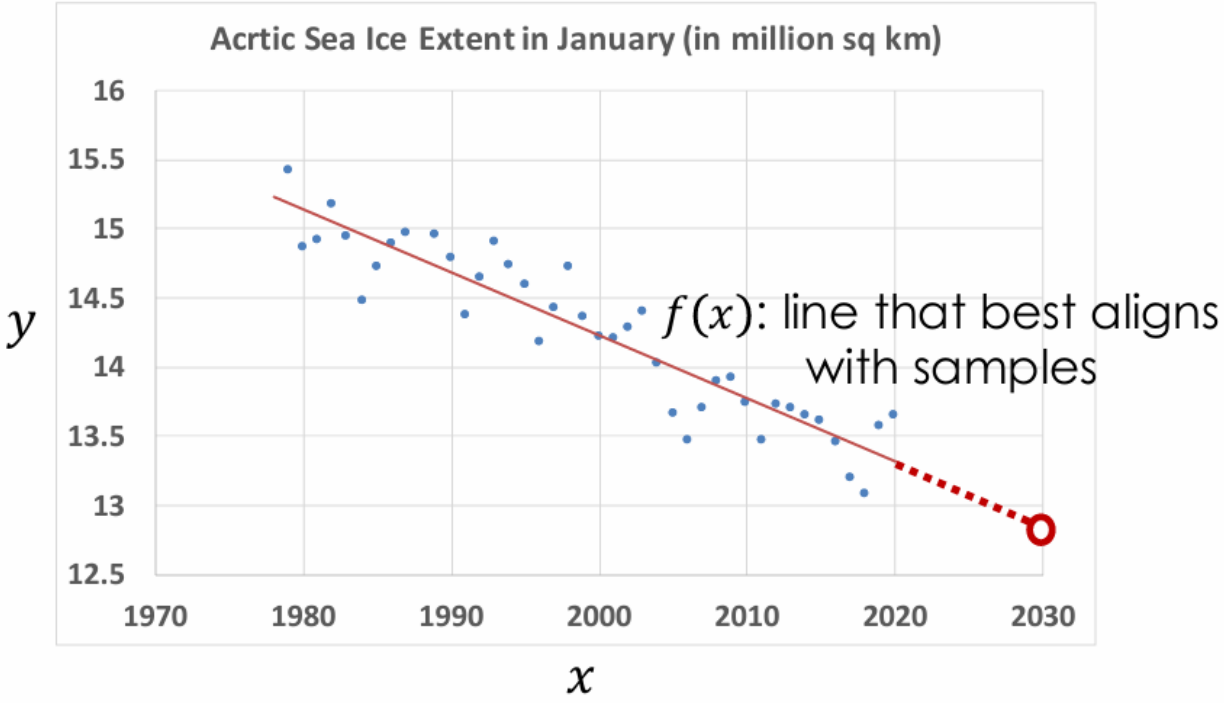


© Copyright National University of Singapore. All Rights Reserved.

The Two Categories of Learning Tasks

- Formulating the regression and classification tasks:
  - Regression: Given a set of data $(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_N, y_N)$ where $y_1, y_2, \ldots, y_N$ are continuous, learn a function $f(\boldsymbol{x})$ to predict real-valued y given $\boldsymbol{x}$.

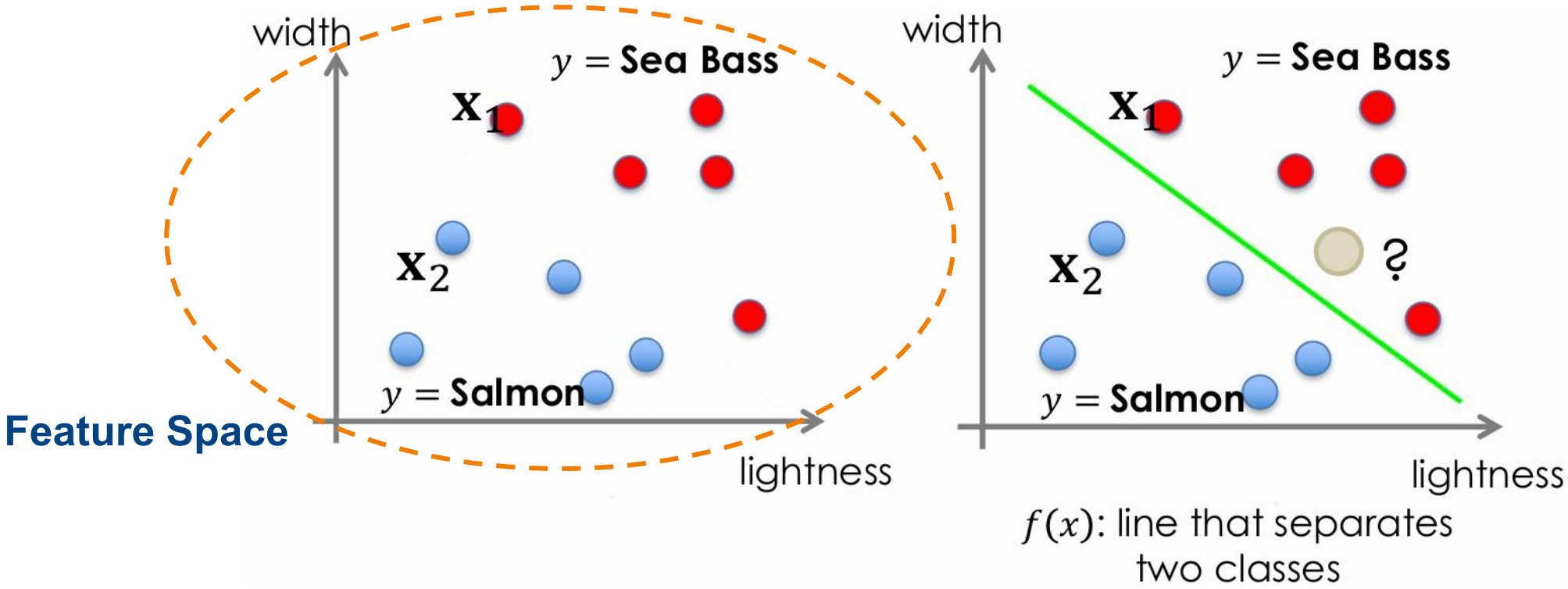




© Copyright National University of Singapore. All Rights Reserved.

The Two Categories of Learning Tasks

- Formulating the regression and classification tasks:
  - Classification: Given a set of data $(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_N, y_N)$ where $y_1, y_2, \ldots, y_N$ are discrete, learn a function $f(\boldsymbol{x})$ to predict categorical y given $\boldsymbol{x}$.



© Copyright National University of Singapore. All Rights Reserved.

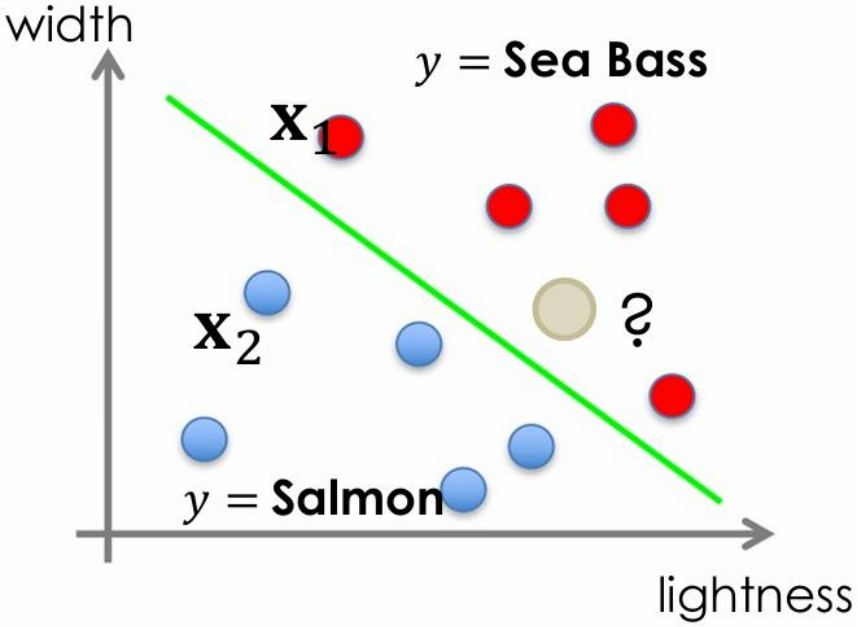
Support Vector Machine 1:

- ✓ Recap on Linear Classifier
- ✓ Classification with Margins: SVM
- ✓ The “Vanilla” SVM: Hard-Margin SVM

© Copyright National University of Singapore. All Rights Reserved.

Linear Methods for Classification

- Look back at the example of classifying Salmon vs. Sea Bass.
  - What is $f(\boldsymbol{x})$?
- $f(\boldsymbol{x})$ is the "boundary" where we "decide" on which class is the data belongs to:
  - To the right: Sea bass;
  - To the left: Salmon.
- $f(\boldsymbol{x})$ is defined as the decision boundary.
  - Also known as the discriminant function.
  - In a high-dimensional feature space, it is also denoted as the decision hyperplane / decision hypersurface.
- How should we construct $f(\boldsymbol{x})$?

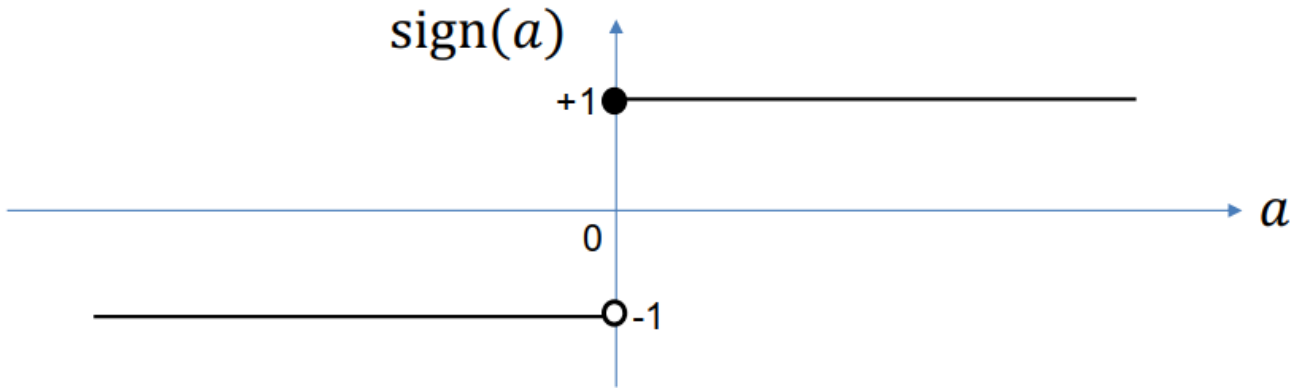


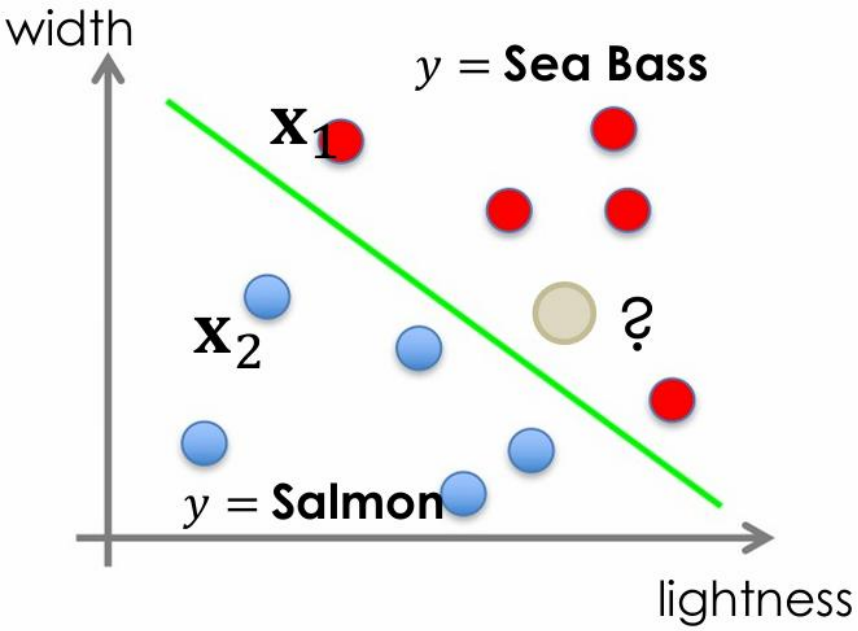
$f(x)$: line that separates two classes

© Copyright National University of Singapore. All Rights Reserved.

Linear Methods for Classification

- To classify between two classes, the easiest model / function we can use is a ***linear*** function – linear classifier (linear regression for classification).
- $f_{\boldsymbol{w},b}(\boldsymbol{x}) = \boldsymbol{w}^T\boldsymbol{x} + b$:
  - For a binary classification task, we compute $f(\boldsymbol{x})$ and classify by looking at if the computed result is positive or negative, thus the discriminant function is best written as:
  - $f(x) = \text{sign}(\boldsymbol{w}^T\boldsymbol{x} + b)$.





$f(x)$: line that separates two classes

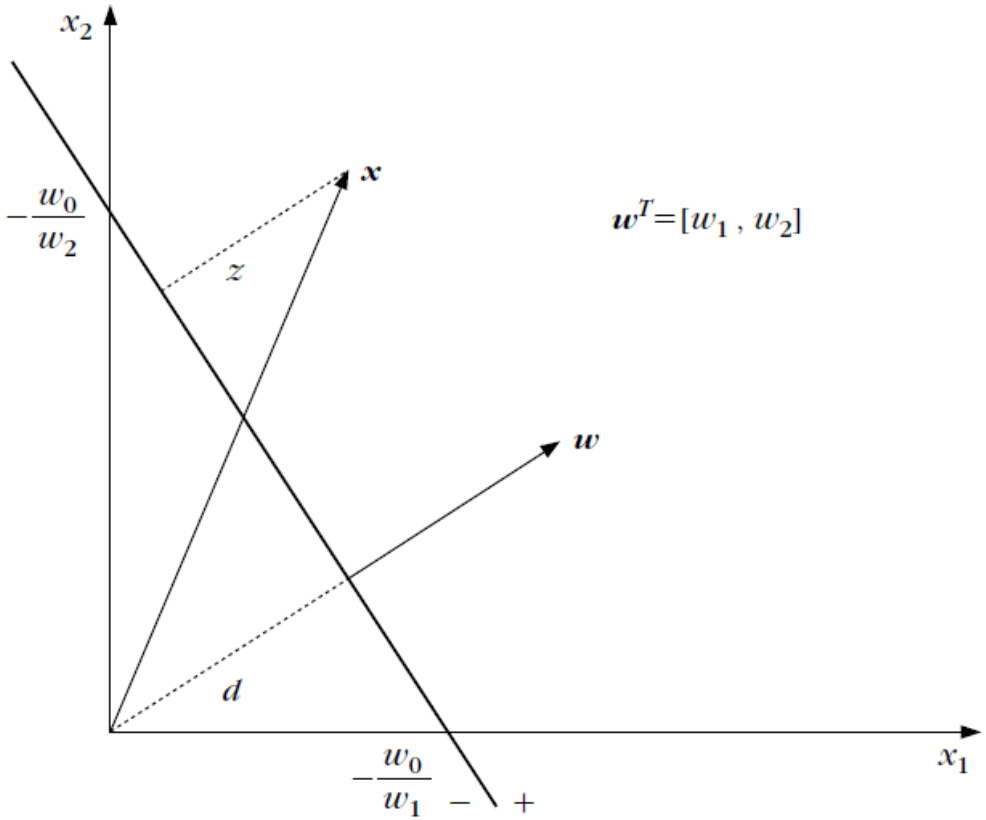
© Copyright National University of Singapore. All Rights Reserved.

Linear Methods for Classification

Class labels:

- +1
- -1

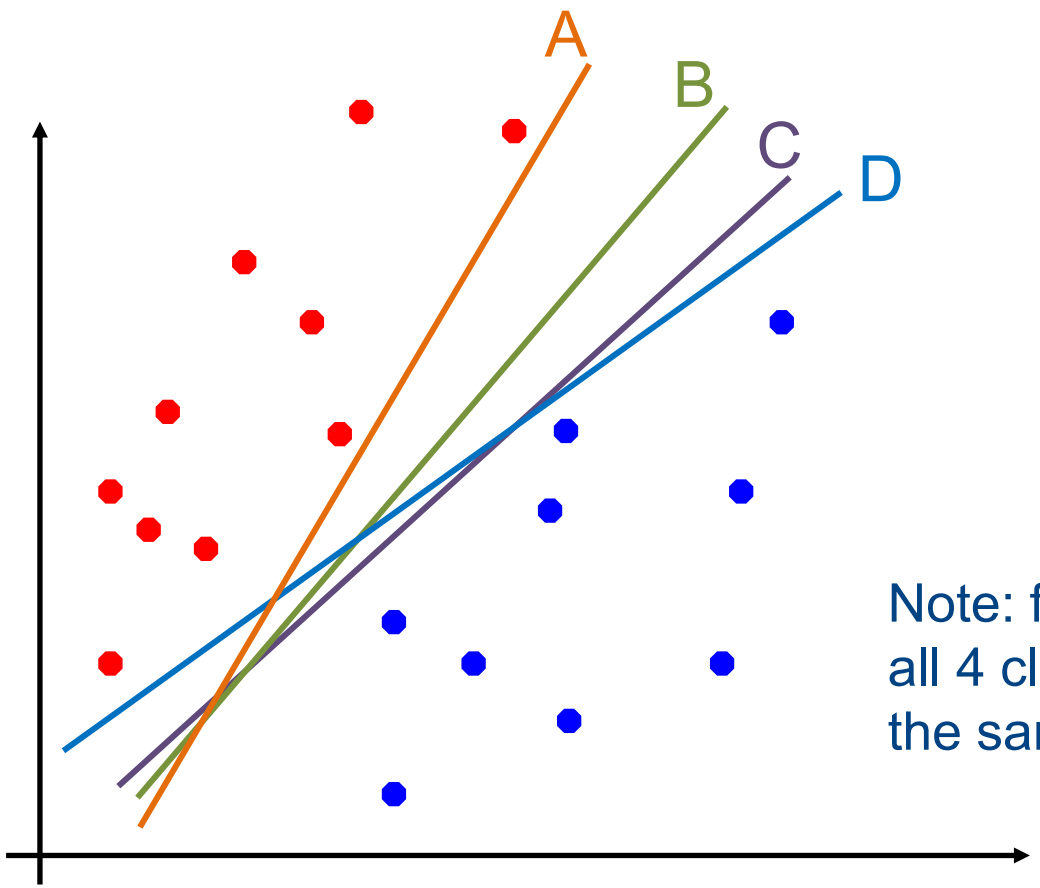
- $f(x) = \text{sign}(\boldsymbol{w}^T \boldsymbol{x} + b)$.
  - What is the direction of $\boldsymbol{w}$?
  - Observe the following equation
  - $0 = \boldsymbol{w}^T x_1 + w_0 = \boldsymbol{w}^T x_2 + w_0 \Rightarrow \boldsymbol{w}^T (x_1 - x_2) = 0$
  - What can we infer from this equation?
- Observe: the difference vector $x_1 - x_2$ lies on the decision hyperplane.
  - The weight vector $\boldsymbol{w}$ must be **orthogonal** to the decision boundary, with the direction pointing towards positive class.



© Copyright National University of Singapore. All Rights Reserved.

What is the Best Linear Classifier?

- Equivalently, how to compare between all these linear classifiers?

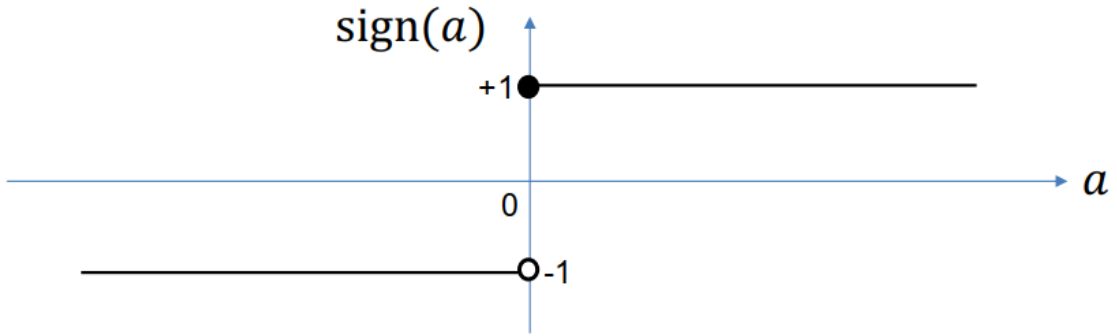


Note: from accuracy perspective, all 4 classifiers' performance are the same: accuracy = 100%!

© Copyright National University of Singapore. All Rights Reserved.

What is the Best Linear Classifier?

- Looking back at the formulation of classification: $f(x) = \mathrm{sign}(\boldsymbol{w}^T\boldsymbol{x} + b)$
  - How can we differentiate between the performance of classification for two data?
  - Apparently, the $\mathrm{sign}(\cdot)$ is the problem that does not work well.



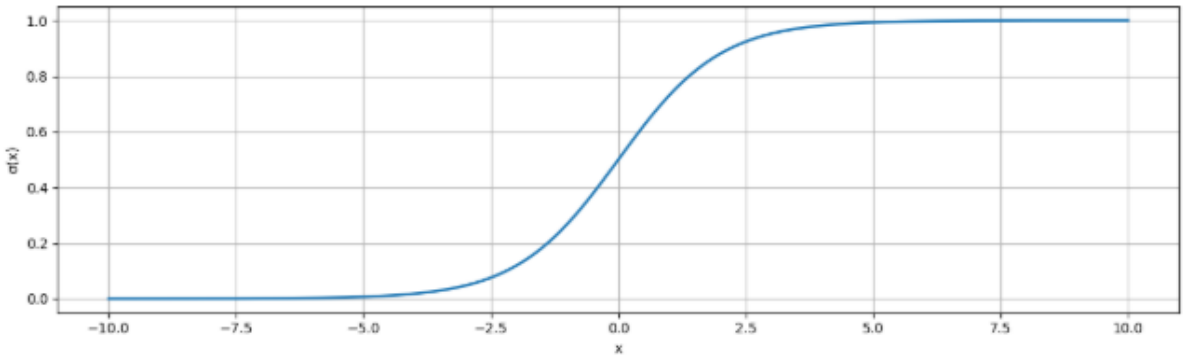
  - In actual implementation, usually we output not the direct class, but the ***probability*** of each class
  - How can we convert the classification result to probability?

© Copyright National University of Singapore. All Rights Reserved.

What is the Best Linear Classifier?

- Looking back at the formulation of classification: $f(x) = \text{sign}(\boldsymbol{w}^T\boldsymbol{x} + b)$
  - Set $s(x) = \boldsymbol{w}^T\boldsymbol{x} + b$, convert $s(x)$ to probability by employing the sigmoid function $\sigma(\cdot)$.

$$p_i(y = 1|x) = \sigma\big(s(x)\big) = \frac{1}{1 + \exp\big(-s(x)\big)}$$



  - To train the linear classifier is simply to choose the best w and b that minimizes a loss defined as:

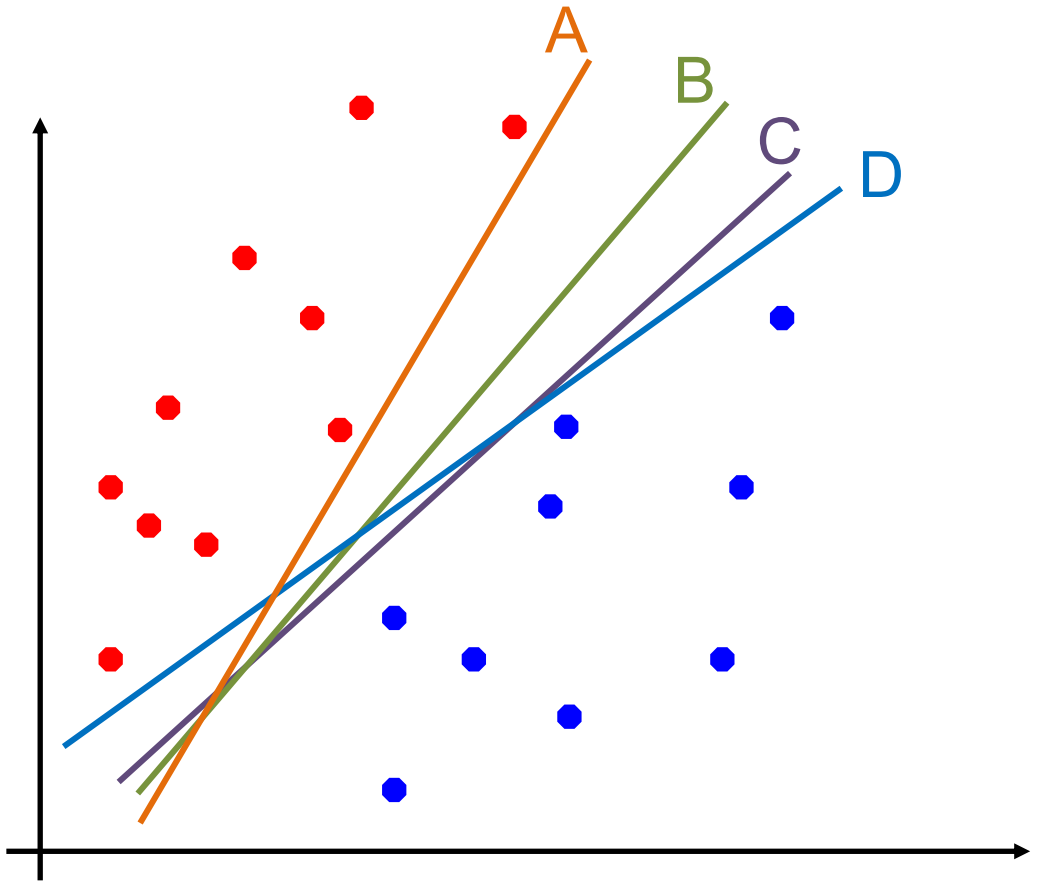
$$\mathcal{L} = \sum_i (y_i \log p_i + (1 - y_i)\log(1 - p_i))$$

Cross-entropy Loss

© Copyright National University of Singapore. All Rights Reserved.

What is the Best Linear Classifier?

- In summary – a better linear classifier should produce a definitive probability for the input data, and result in a low cross-entropy loss.
- Question: what would such a linear classifier be like geometrically?
- Which one of the four classifiers has the lowest cross-entropy loss?
  - Indeed, very difficult to tell directly. How about comparing A vs. B first.
  - Looking back at the score function, how can we further interpret $s(\cdot)$?
  - The larger the score $s(\cdot)$ is, the more **confident** we are in the classification!



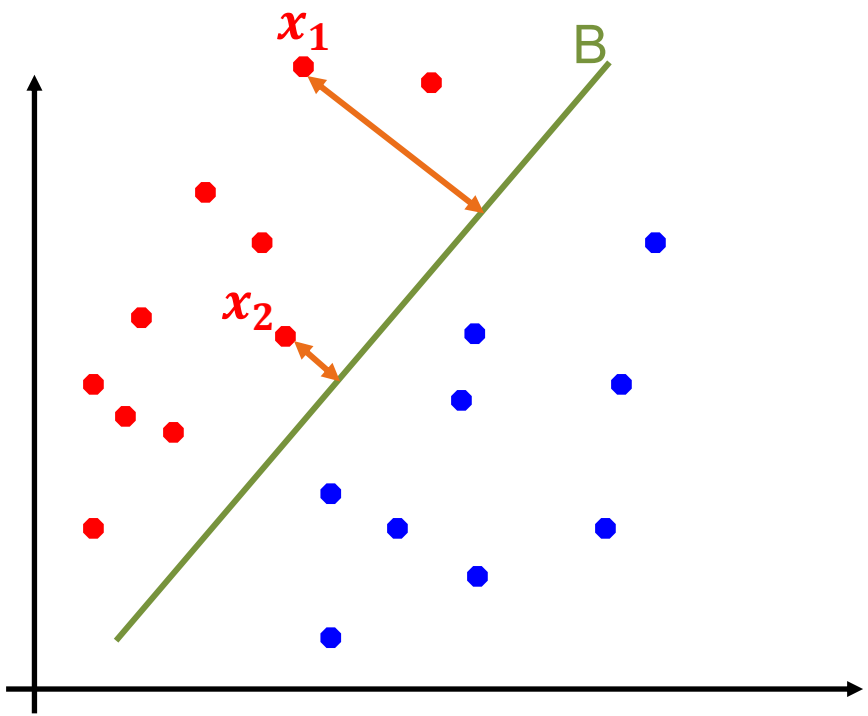
© Copyright National University of Singapore. All Rights Reserved.

Better Linear Classifiers with Margins

- Comparing x_1 with x_2, apparently x_1 can be classified more confidently than x_2 since it is further away from the decision boundary.
  - Margin – the "distance" from the data to the boundary. How to consider correctness?
  - Signed margin – positive being correctly classified and negative being misclassified.

$$m(x, y) = ys(x)$$

  - Why do we want a larger margin (signed margin?)
  - Comparing the classification of x_1 and x_2, which one is more robust to perturbations?
  - If a point is very close to the boundary, a small amount of noise could flip its label. If it's far away, it's more stable.
  - Goal: building a linear classifier where the data are classified with larger margins.



© Copyright National University of Singapore. All Rights Reserved.

Better Linear Classifier: Support Vector Machine (SVM)

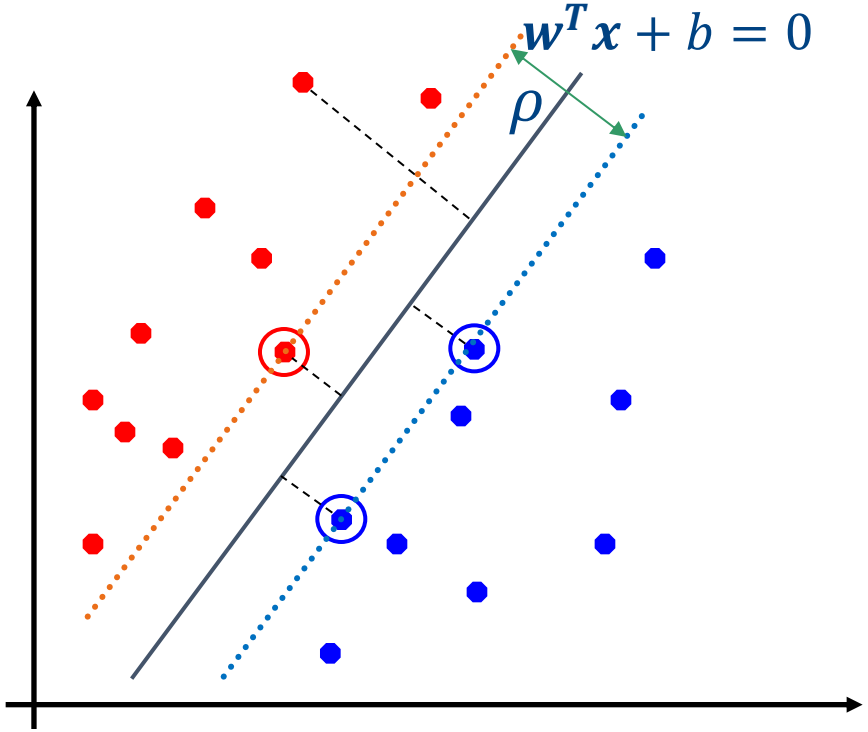
- Formally, how can we find the better linear classifier(s)?
- We find the better linear classifier following the SVM principle: choosing w and b that maximize the minimum margin over training examples.

$$\max_{w,b} \min_{i} \frac{y_i(w^T x_i + b)}{\|w\|}$$

- Or in an objective-constraint fashion:

$$\min_{w,b} \frac{1}{2}\|w\|^2 \left(\text{or } \frac{1}{2} w^T w\right), \text{s.t. } y_i(w^T x_i + b) \geq 1$$

- Let us derive these objectives in detail.



© Copyright National University of Singapore. All Rights Reserved.

SVM: Maximizing the Margin

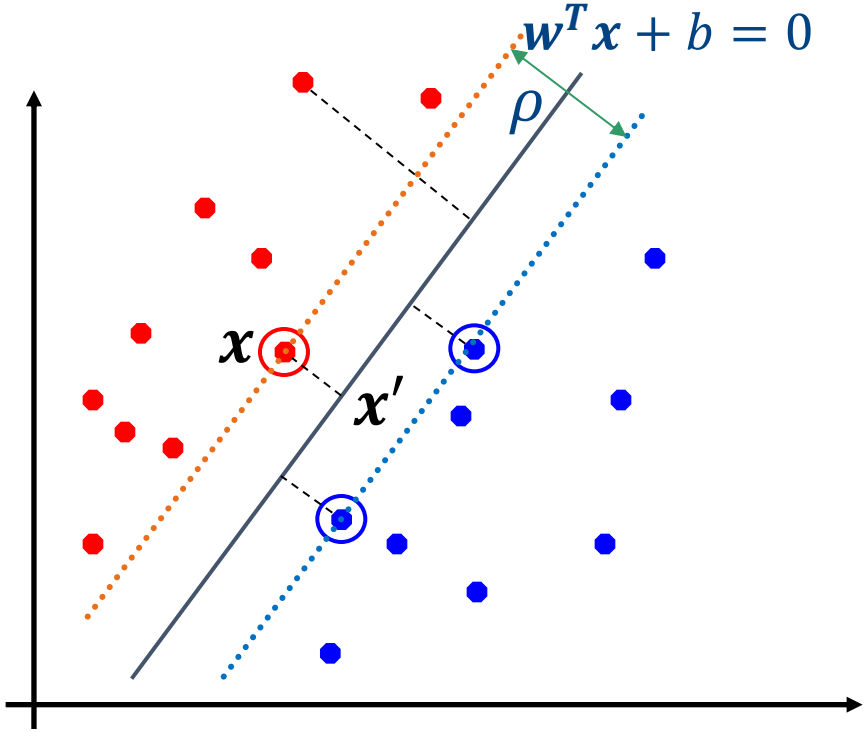
- As shown, the orange and blue dashed lines are where the points with minimum margin to the decision boundary.
- How do we express the orange and blue dashed lines?

Orange dashed line:

$$\begin{cases} \boldsymbol{w}^{\boldsymbol{T}}\boldsymbol{x}' + b = 0 \\ \dfrac{\boldsymbol{x} - \boldsymbol{x}'}{\rho/2} = \dfrac{\boldsymbol{w}}{\|\boldsymbol{w}\|} \rightarrow \boldsymbol{x}' = \boldsymbol{x} - \dfrac{\rho \boldsymbol{w}}{2\|\boldsymbol{w}\|} \end{cases}$$

$$\boldsymbol{w}^T\left(\boldsymbol{x} - \frac{\rho \boldsymbol{w}}{2\|\boldsymbol{w}\|}\right) + b = 0$$

$$\boldsymbol{w}^T\boldsymbol{x} + b = \frac{\rho\|\boldsymbol{w}\|}{2}$$



© Copyright National University of Singapore. All Rights Reserved.

SVM: Maximizing the Margin

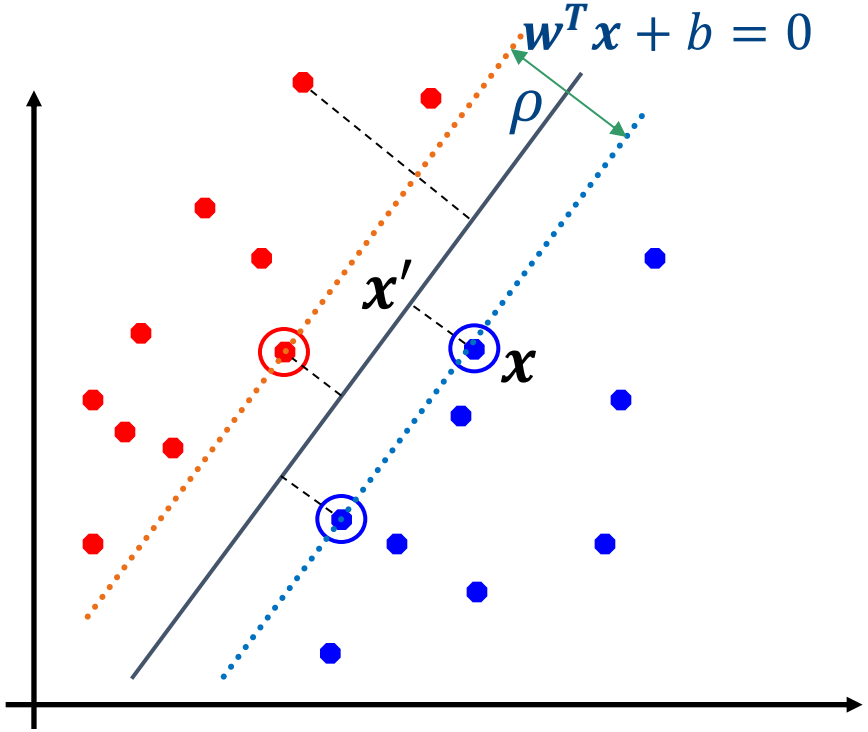
- As shown, the orange and blue dashed lines are where the points with minimum margin to the decision boundary.
- How do we express the orange and blue dashed lines?

Blue dashed line:

$$\begin{cases} \boldsymbol{w}^{\boldsymbol{T}}\boldsymbol{x}' + b = 0 \\ \dfrac{\boldsymbol{x} - \boldsymbol{x}'}{\rho/2} = -\dfrac{\boldsymbol{w}}{\|\boldsymbol{w}\|} \rightarrow \boldsymbol{x}' = \boldsymbol{x} + \dfrac{\rho \boldsymbol{w}}{2\|\boldsymbol{w}\|} \end{cases}$$

$$\boldsymbol{w}^T\left(\boldsymbol{x} - \frac{\rho \boldsymbol{w}}{2\|\boldsymbol{w}\|}\right) + b = 0$$

$$\boldsymbol{w}^T\boldsymbol{x} + b = -\frac{\rho\|\boldsymbol{w}\|}{2}$$



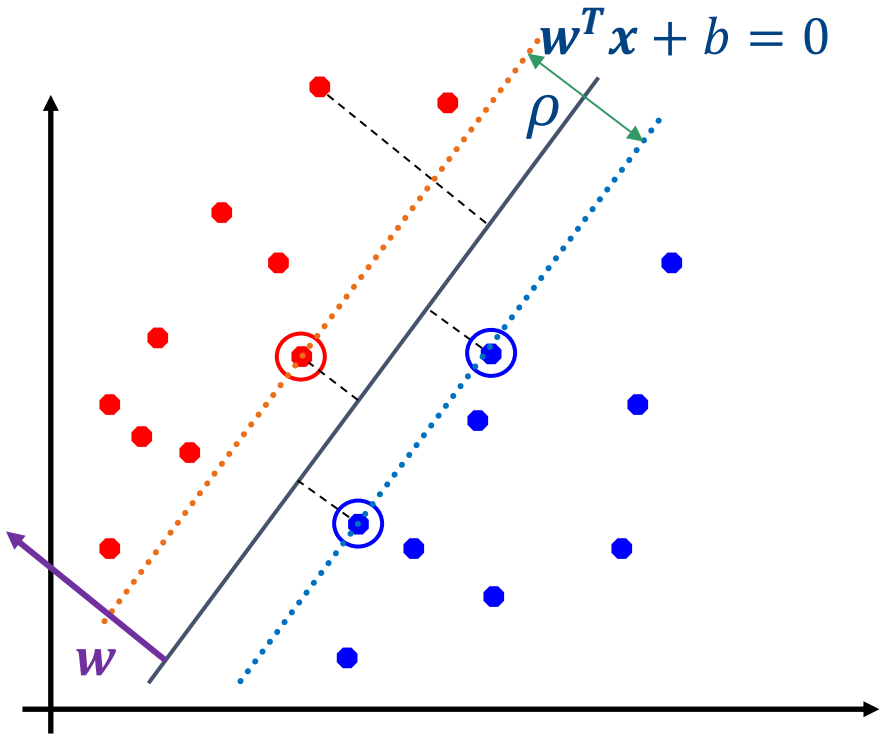
© Copyright National University of Singapore. All Rights Reserved.

SVM: Maximizing the Margin

- By collating the orange and blue dashed lines, we have:
- Plus Plane: $\boldsymbol{w}^T\boldsymbol{x} + b = 1$
- Minus Plane: $\boldsymbol{w}^T\boldsymbol{x} + b = -1$

$$\rho = \frac{2}{\|\boldsymbol{w}\|}$$

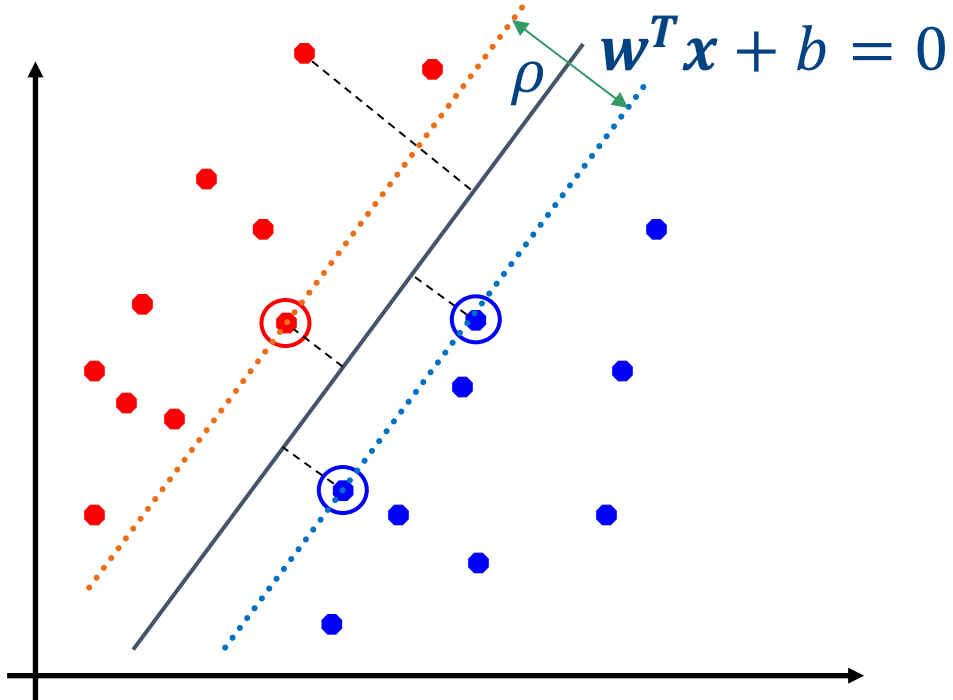
  - As observed, we assume the positive class is of label $+1$ and the negative class is of label -1.
  - The circled red and blue points are the data points closest to the decision boundary and are defined as ***support vectors***.



© Copyright National University of Singapore. All Rights Reserved.

SVM: Maximizing the Margin

- The objective is thus to find w, b such that:
  - All positive training points $(\boldsymbol{x}_i, y_i = 1)$ in the red zone ➡ $\boldsymbol{w}^T\boldsymbol{x}_i + b \geq 1, \text{if } y_i = 1$
  - All negative training points $(\boldsymbol{x}_i, y_i = -1)$ in the blue zone ➡ $\boldsymbol{w}^T\boldsymbol{x}_i + b \leq -1, \text{if } y_i = -1$
  - The margin ρ is maximized ➡ $\max \frac{2}{\|\boldsymbol{w}\|}$



© Copyright National University of Singapore. All Rights Reserved.

SVM Primal as Quadratic Program

- From $\boldsymbol{w}^T\boldsymbol{x}_i + b \geq 1, \text{if } y_i = 1$ and $\boldsymbol{w}^T\boldsymbol{x}_i + b \leq -1, \text{if } y_i = -1$, we get the constraint of the SVM principle: $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1$
- From the objective $\max \frac{2}{\|\boldsymbol{w}\|}$, we can reformulate as a minimization objective (as in training for linear regression): $\min \|w\|$
- Or more often $\min \|w\|^2$ to ensure positivity, or even more often $\min \frac{1}{2} w^T w$ such that during differentiation, the scaling factor is 1.
- We thus formally formulate the SVM as an **optimization** problem:

$$\min_{w,b} \frac{1}{2} w^T w \,, \text{s.t.}\, y_i(w^T x_i + b) \geq 1$$

© Copyright National University of Singapore. All Rights Reserved.

SVM Primal as Quadratic Program

$$\min_{w,b} \frac{1}{2} w^T w \text{ , s.t. } y_i(w^T x_i + b) \geq 1, \forall i$$

- This is a form of a quadratic program (QP). What is a quadratic program?
- We want to choose some numbers (decision variables) to make a “score” as small as possible.
- The score is shaped like a smooth bowl (so there is usually a clear “best” point). (Mathematically, being quadratic instead of linear)
- We must also satisfy constraints (rules/limits).

- For SVM, there should be a definite solution as this problem is a convex problem.

© Copyright National University of Singapore. All Rights Reserved.

SVM Primal (Primal Form of SVM)

$$\min_{w,b} \frac{1}{2} w^T w \,, \text{s.t.}\, y_i(w^T x_i + b) \geq 1, \forall i$$

- This form expressed as a QP is denoted as the **primal** form of SVM.
- Primal: a more fundamental, original state of the problem.
- It is a primal for because it is the optimization written directly in terms of the original model parameters:
  - Variables: w and b;
  - Constraints directly enforce margins on training points.

- Very direct and intuitive. Any other pros or limitations?

© Copyright National University of Singapore. All Rights Reserved.

SVM Primal: Pros and Limitations

- Suppose the data is d-dimension, then SVM primal would require to solve $d+1$ variables under N constraints.
  - For linear data or data in low dimensions, training SVM in its primal form is excellent.
- It is impossible to write "subject to xxx" in coding and expect the program to follow the constraints.
  - Solution: re-write the "subject to" phrase as a part of the objective function:
  - $\min_{w,b} \frac{1}{2} w^T w + C \sum_i \max\left(0,\ 1 - y_i(w^T x_i + b)\right)$
  - Usually, we don't directly train with x_i directly, but with an explicit feature vector $\phi(x_i)$.
  - This creates a nonlinear decision boundary in the high-dimensional feature space, thus w becomes impractical to compute and implement directly.
- Any alternatives to the primal form which can work for high-dimensional data?

© Copyright National University of Singapore. All Rights Reserved.

The Lagrangian Function and KKT Conditions

- Looking back at the primal form of SVM, the problem we are solving are of form: Minimizing a cost ***while*** obeying constraints (limits / rules).
- Primal form: objective & constraints are considered separately
- Lagrangian idea: combine objective & constraints together:
  - If a constraint is violated, add a **penalty**.
  - The **Lagrange multipliers** are like *automatic penalty weights* learned at the optimum.
- Formally, the Lagrangian function can be generally formulated as:

$$L(x, \lambda, \mu) = f(x) + \lambda^T(\text{inequalities}) + \mu^T(\text{equalities})$$

  - μ: adjusts how much we care about satisfying equalities.
  - λ: adjusts how much inequality limits “push back”.

© Copyright National University of Singapore. All Rights Reserved.

The Lagrangian Function and KKT Conditions

- For the Lagrangian function, what would an optimal solution x^* hold?
  - **Feasibility (obey the rules):** x^* satisfies all constraints.
  - **Stationarity (no improving direction once penalties included)**: cannot tweak x a tiny bit to improve the objective **without** immediately paying through constraints.
  - **Nonnegativity (inequality penalties can't be negative)**: $\lambda \geq 0$, Inequality constraints only "push back" when you try to violate them.
  - **Complementary slackness (active constraints are the only ones that matter)**: For each inequality constraint:
  - Either it is **active/tight** (right on the boundary) $\rightarrow$ it can have a **nonzero push** ($\lambda > 0$)
  - Or it is **inactive/loose** (margin left) $\rightarrow$ it exerts **no push** ($\lambda = 0$)

- These are summarized as the Karush-Kuhn-Tucker (KKT) conditions.

© Copyright National University of Singapore. All Rights Reserved.

SVM Dual with KKT Condition

- Now, with our understanding of the Lagrangian function and the KKT conditions, let us formulate the dual form of SVM.
- Key components: $f(\cdot)$ – the objective function; $g(\cdot)$ – the inequality penalty.
  - Note: we do not have an equality reward function as we don't care about equality.
  - Define the Lagrangian functions with:

$$f(\boldsymbol{w}, b) = \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w}, \qquad g_i(\boldsymbol{w}, b) = y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) - 1$$

- The Lagrangian function is thus:

$$\mathcal{L}(\boldsymbol{w}, b, \boldsymbol{\lambda}) = f(\boldsymbol{w}, b) - \sum_i \lambda_i g_i(\boldsymbol{w}, b)\,, \text{where } \lambda_i \geq 0, \forall i$$

- Question: why do we **minus** the inequality penalty instead of plus?

© Copyright National University of Singapore. All Rights Reserved.

SVM Dual with KKT Condition

- The Lagrangian function:

$$\mathcal{L}(\boldsymbol{w}, b, \boldsymbol{\lambda}) = f(\boldsymbol{w}, b) - \sum_i \lambda_i g_i(\boldsymbol{w}, b) = \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} - \sum_i \lambda_i (y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) - 1\ ), \text{where } \lambda_i \geq 0, \forall i$$

- The goal of the Lagrangian function is to penalize by increasing the value of the objective function when the inequality is **NOT** satisfied.
- $g_i(\boldsymbol{w}, b) = y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) - 1$: the penalty should be imposed when g is negative (violating the constraint).
- Since λ must be non-negative, the only way to achieve this is to perform a subtraction instead of an addition.

© Copyright National University of Singapore. All Rights Reserved.

SVM Dual with KKT Condition

- The Lagrangian function:

$$\mathcal{L}(\boldsymbol{w}, b, \boldsymbol{\lambda}) = f(\boldsymbol{w}, b) - \sum_i \lambda_i g_i(\boldsymbol{w}, b) = \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} - \sum_i \lambda_i (y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) - 1\ ), \text{where } \lambda_i \geq 0, \forall i$$

- Next, we look at the KKT conditions for optimality:
  - A. Stationarity: gradient of Lagrangian is 0.

$$\begin{cases} \dfrac{\partial \mathcal{L}}{\partial w^*} = 0 \ \rightarrow w^* - \sum_i \lambda_i y_i x_i = 0 \ \rightarrow w^* = \sum_i \lambda_i y_i x_i \\ \dfrac{\partial \mathcal{L}}{\partial b^*} = 0 \ \rightarrow \sum_i \lambda_i y_i = 0 \end{cases}$$

  - B. Primal Feasibility: the constraints in the primal problem should be satisfied.

$$g_i(\boldsymbol{w}^*, b^*) = y_i(\boldsymbol{w}^{*T}\boldsymbol{x}_i + b^*) - 1 \geq 0, \forall i$$

© Copyright National University of Singapore. All Rights Reserved.

SVM Dual with KKT Condition

- The Lagrangian function:

$$\mathcal{L}(\boldsymbol{w}, b, \boldsymbol{\lambda}) = f(\boldsymbol{w}, b) - \sum_i \lambda_i g_i(\boldsymbol{w}, b) = \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} - \sum_i \lambda_i (y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) - 1\ ), \text{where } \lambda_i \geq 0, \forall i$$

- Next, we look at the KKT conditions for optimality:
  - C. Dual Feasibility: constraints in the dual problem should be satisfied.

$$\lambda_i \geq 0, \forall i$$

  - D. Complementary Slackness: the Lagrangian multiplier (lambda) and constraint must be complementary. I.e., either constraint active or lambda equal to zero under **optimal** solution.

$$\lambda_i g_i(\boldsymbol{w}^*, b^*) = 0, \forall i$$

$$\begin{cases} \text{if } g_i(\boldsymbol{w}^*, b^*) > 0 \text{ (inactive)}, \lambda_i = 0 \\ \text{ if } g_i(\boldsymbol{w}^*, b^*) = 0 \text{ (active)}, \lambda_i > 0 \end{cases}$$

  - Under optimal solution, no data should break the constraints.

© Copyright National University of Singapore. All Rights Reserved.

Question: how to derive b^*?

Deriving the SVM Dual Form

- Now, let us formally derive the SVM dual form:

$$\min_{w,b} \mathcal{L}(\boldsymbol{w}, b, \boldsymbol{\lambda}) = \min_{w,b} f(\boldsymbol{w}, b) - \sum_i \lambda_i g_i(\boldsymbol{w}, b)$$

$$= \min_{w,b} \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} - \sum_i \lambda_i (y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) - 1)$$

$$= \min_{w,b} \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} - \sum_i \lambda_i y_i \boldsymbol{w}^T\boldsymbol{x}_i - \sum_i \lambda_i y_i b + \sum_i \lambda_i$$

$$= \frac{1}{2}\boldsymbol{w}^{*T}\boldsymbol{w}^* - \boldsymbol{w}^{*T}\boldsymbol{w}^* + \sum_i \lambda_i = -\frac{1}{2}\sum_i \lambda_i y_i \boldsymbol{x}_i^T \sum_i \lambda_i y_i \boldsymbol{x}_i + \sum_i \lambda_i$$

$$= -\frac{1}{2}\sum_{i,j} \lambda_i \lambda_j y_i y_j \boldsymbol{x}_i^T \boldsymbol{x}_j + \sum_i \lambda_i$$

$$w^* = \sum_i \lambda_i y_i x_i$$

$$b^* = y_s - \sum_i \lambda_i y_i x_i^T x_s$$

With (x_s, y_s) being a support vector point on the margin.

What can we infer from this derivation?

© Copyright National University of Singapore. All Rights Reserved.

SVM Dual

- The primal form and dual form are the same problem, thus:
- The primal form of $\min_{\boldsymbol{w},b} \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w}, s.t. y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1, \forall i$ is equivalent to the dual form of $\max_{\boldsymbol{\lambda}} -\frac{1}{2}\sum_{i,j}\lambda_i\lambda_j y_i y_j \boldsymbol{x}_i^T\boldsymbol{x}_j + \sum_i \lambda_i, s.t.\ \lambda_i \geq 0,\ \sum_i \lambda_i y_i = 0, \forall i$
- !!Notice anything?
- Why does the primal form of min become a max in the dual form?
- Set $h(\lambda) = \min_{w,b} \mathcal{L}(\boldsymbol{w}, b, \boldsymbol{\lambda})$, $\forall \boldsymbol{\lambda} \geq 0$, $h(\boldsymbol{\lambda})$ is a **lower bound** on the primal optimum.
- We aim for the best (largest) lower bound: $\max_{\boldsymbol{\lambda} \geq \mathbf{0}} h(\lambda)$
- The clean reason “min in primal” becomes “max in dual.”

© Copyright National University of Singapore. All Rights Reserved.

SVM Dual: Summary

- What can we infer from our formulation and derivation?
- From $w^* = \sum_i \lambda_i y_i x_i$, we can infer that w^* lies in the span of the training data – i.e., the solution is built from data points.
- The dual has one variable (λ_i) per training point, but **most** λ_i become 0 (from complementary slackness). Points with $\lambda_i > 0$ are the support vectors, where the prediction depends only on them:

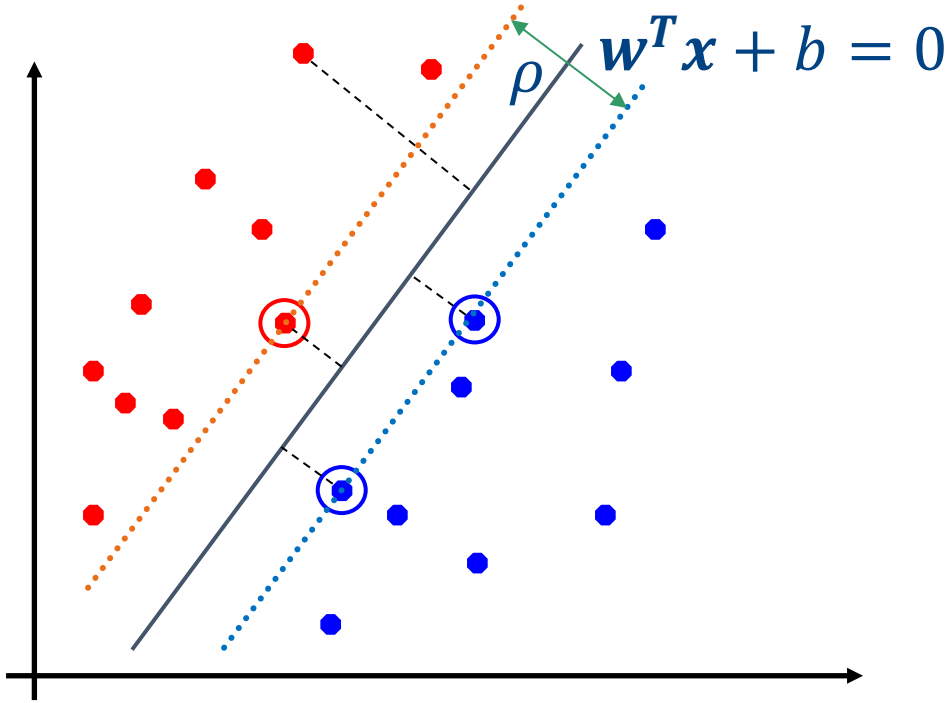
$$f(x) = \text{sign}\left(\sum_{i \in SV} \lambda_i y_i x_i^T x + b\right)$$

- Open question:

Prove: $\min_{\boldsymbol{w},b} f(\boldsymbol{w}, b)\,, s.t.\, g_i(\boldsymbol{w}, b) \geq 0$ $=$ $\max_{\boldsymbol{\lambda}} \min_{\boldsymbol{w},b} \mathcal{L}(\boldsymbol{w}, b, \boldsymbol{\lambda})$ s.t. $\lambda_i \geq 0$

© Copyright National University of Singapore. All Rights Reserved.

SVM Dual: Summary



- We can thus summarize a final time the optimum weight and bias from the dual form.

$$w^* = \sum_i \lambda_i y_i x_i$$

$$b^* = y_s - \sum_i \lambda_i y_i x_i^T x_s = \frac{1}{N_s} \sum \left(y_s - \sum_i \lambda_i y_i \boldsymbol{x}_i^T \boldsymbol{x}_s \right)$$

- With (x_s, y_s) being a support vector point on the margin.
- The SVM is only relevant to the support vectors (considered minimal w.r.t. the training data).

© Copyright National University of Singapore. All Rights Reserved.

Quick Review Question

For SVM with the following objective function:

$$\min_{\boldsymbol{w},b} \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w}$$

s.t. $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1, \forall i$

or

$$\max_{\boldsymbol{\lambda}} -\frac{1}{2}\sum_{i,j} \lambda_i\lambda_j y_i y_j \boldsymbol{x}_i^T\boldsymbol{x}_j + \sum_i \lambda_i$$

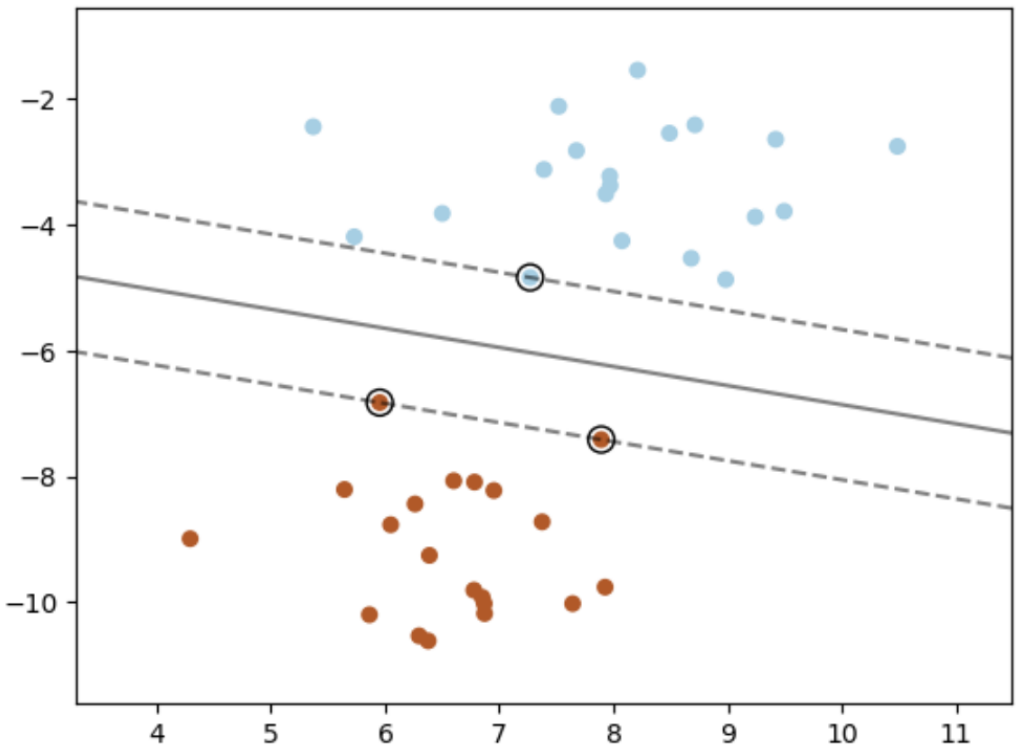
s.t. $\lambda_i \geq 0, \sum_i \lambda_i y_i = 0, \forall i$

What are the support vectors?

A. Data points that lie exactly on the margin boundaries.
B. Data points that have $\lambda_i > 0$.
C. Data points that determine the decision boundary
D. Data points inside the margin
E. Data points that are misclassified

© Copyright National University of Singapore. All Rights Reserved.

Demo: Hard-Margin SVM with Scikit-Learn



© Copyright National University of Singapore. All Rights Reserved.

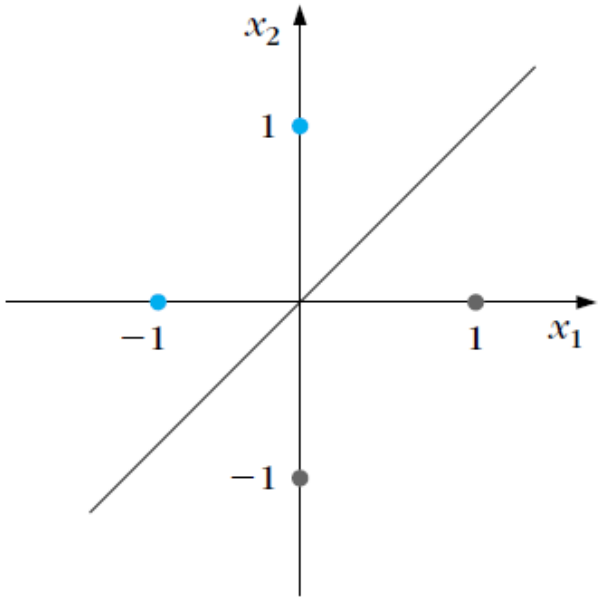
Support Vector Machine 2:

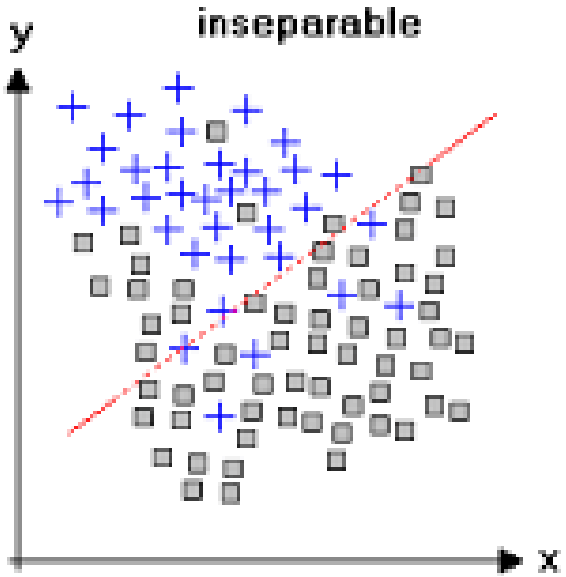
- ✓ Soft-Margin SVM
- ✓ Kernel SVM
- ✓ Multi-Class SVM

© Copyright National University of Singapore. All Rights Reserved.

SVM: Linearly Inseparable Data

- Recall, for SVM, our goal: building a linear classifier where the data are classified with larger margins.
- What are the limitations for our discussions so far?
- We always assume that the data are linearly separable – i.e., there is always at least ONE definite (set of) linear function(s) that can separate the data.
- Hard-Margin SVM.
- In many cases, the classes are not linearly separable!
- Can we still leverage on linear classifiers as SVMs for a decent performance in classification?

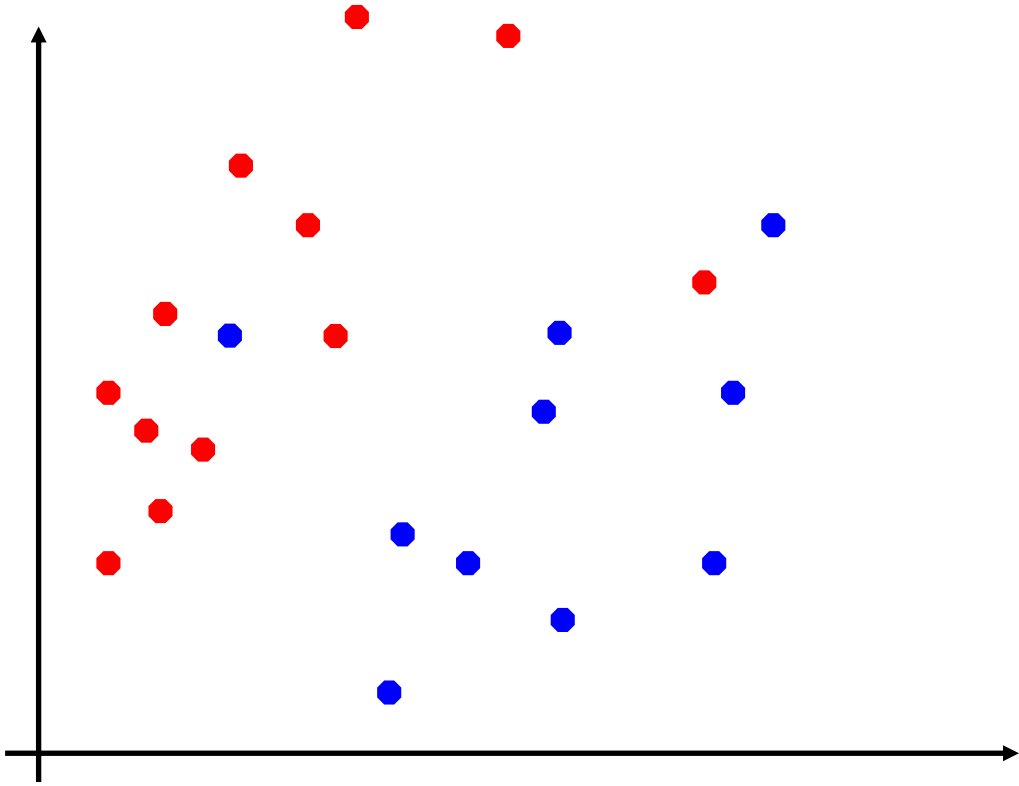




© Copyright National University of Singapore. All Rights Reserved.

SVM: Linearly Inseparable Data

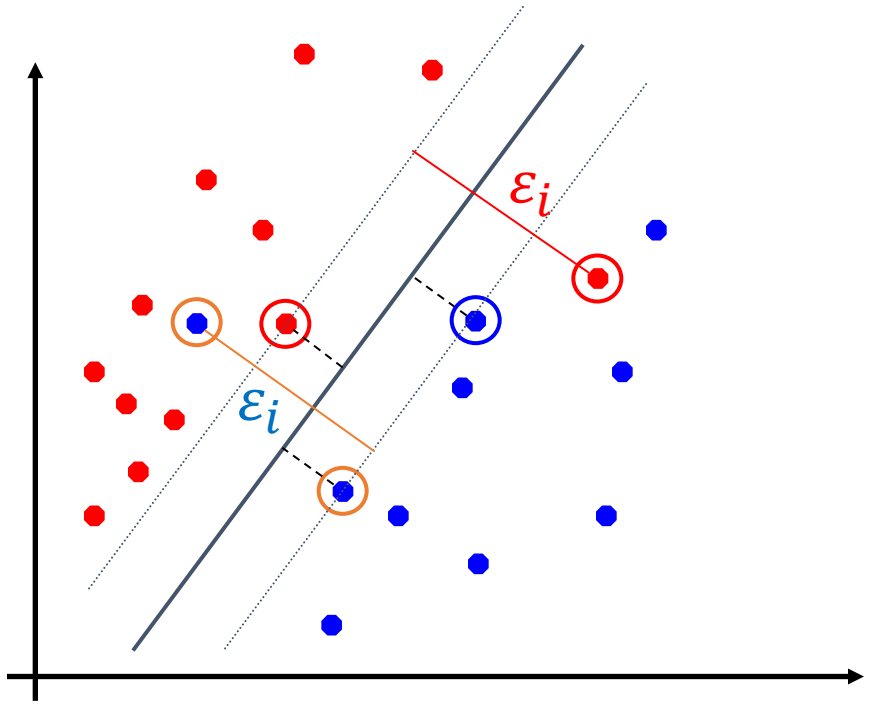
- No longer able to force the constraint in SVM for **EVERY** data point.
- Question becomes what are the compromises we would take or what extra effort we would take to transfer the problem into our familiar SVM with hard-margin SVM?
- Trick 1: allow a (reasonably) few points to violate the margin – relax the constraint: Soft-Margin SVM.
- Trick 2: map data to a higher dimensional space using a function (kernel function) and perform linear classification in the higher-dimension space: Kernel SVM.



© Copyright National University of Singapore. All Rights Reserved.

Soft-Margin SVM: Relaxing Constraints

- Trick 1: allow a (reasonably) few points to violate the margin – relax the constraint: Soft-Margin SVM. How?
- Introducing slack variables ε_i: measures how much a data point violates the margin.
- If $\varepsilon_i = 0$, then $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1$: correctly classified on or outside of the margin.
- If $0 < \varepsilon_i \leq 1$, then $0 \leq y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) < 1$: correctly classified inside the margin.
- If $\varepsilon_i > 1$, then $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) < 0$: misclassified on the wrong side.



ε_i is not the distance!

© Copyright National University of Singapore. All Rights Reserved.

Soft-Margin SVM: Slack Variables

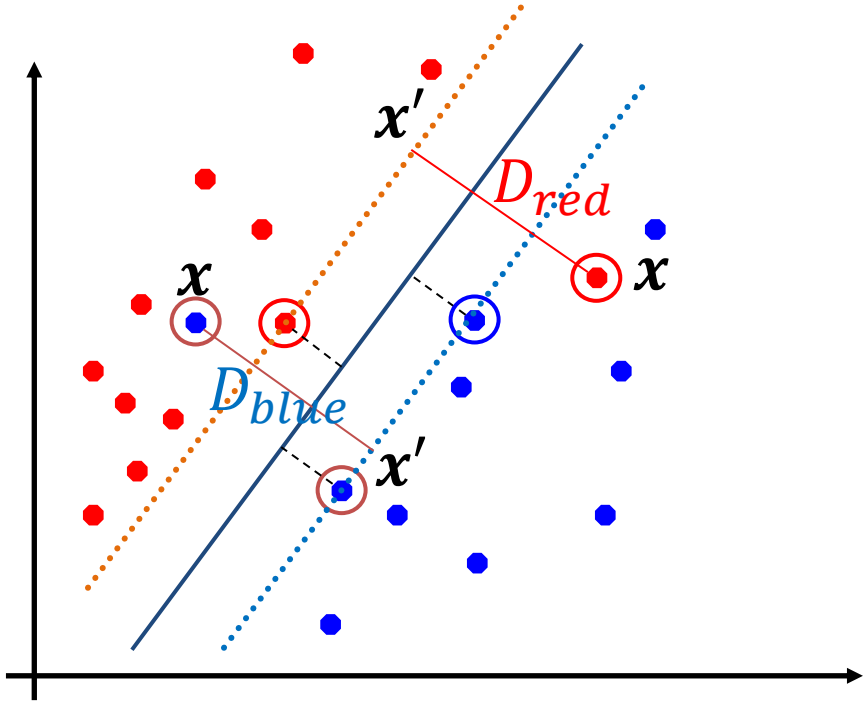
- Note that the slack variables ε_i are not the distances of the misclassified / in-margin data towards the margins.
- Let's see how ε differs from the distance D:
- If $y_i = -1$: (blue)

$$\begin{cases} w^T x' + b = -1 \\ \dfrac{\boldsymbol{x}_i - \boldsymbol{x}'}{D_{blue}} = \dfrac{\boldsymbol{w}}{\|\boldsymbol{w}\|} \end{cases}$$

- If $y_i = 1$: (red)

$$\begin{cases} w^T x' + b = 1 \\ \dfrac{\boldsymbol{x}_i - \boldsymbol{x}'}{D_{red}} = \dfrac{-\boldsymbol{w}}{\|\boldsymbol{w}\|} \end{cases}$$

$$\begin{cases} w^T x' + b = -1 \\ \dfrac{\boldsymbol{x}_i - \boldsymbol{x}'}{D} = \dfrac{-y_i \boldsymbol{w}}{\|\boldsymbol{w}\|} \rightarrow \boldsymbol{x}' = \boldsymbol{x}_i + \dfrac{D y_i \boldsymbol{w}}{\|\boldsymbol{w}\|} \end{cases}$$



© Copyright National University of Singapore. All Rights Reserved.

Soft-Margin SVM: Slack Variables

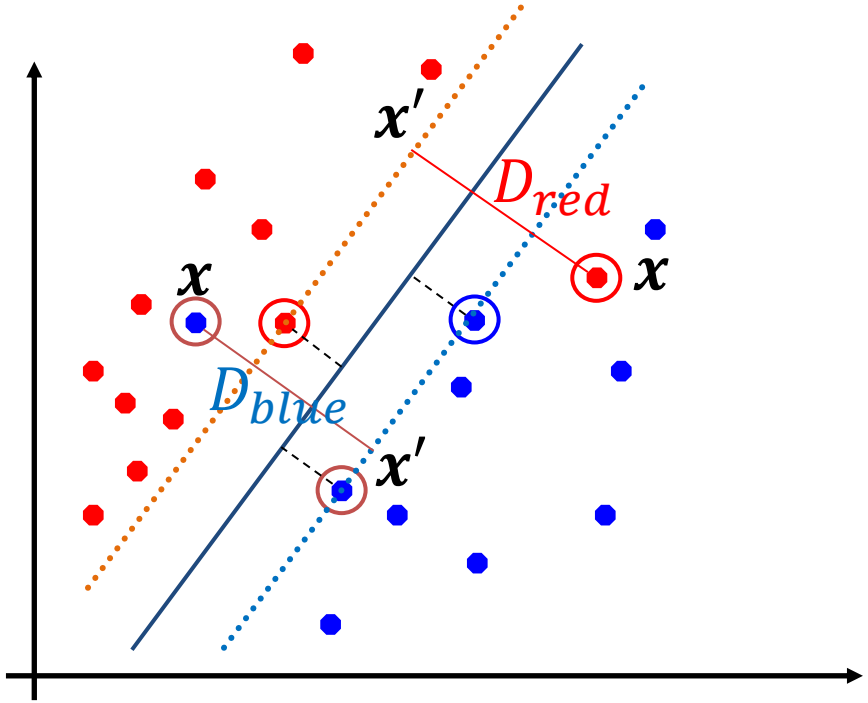
- How ε differs from (and relates to) the distance D:

$$\begin{cases} w^T x' + b = -1 \\ \dfrac{\boldsymbol{x}_i - \boldsymbol{x}'}{D} = \dfrac{-y_i \boldsymbol{w}}{\|\boldsymbol{w}\|} \rightarrow \boldsymbol{x}' = \boldsymbol{x}_i + \dfrac{D y_i \boldsymbol{w}}{\|\boldsymbol{w}\|} \end{cases}$$

- We thus have:

$$\boldsymbol{w}^T \left(\boldsymbol{x}_i + \frac{D y_i \boldsymbol{w}}{\|\boldsymbol{w}\|} \right) + b = y_i \rightarrow D = \frac{y_i - \boldsymbol{w}^T \boldsymbol{x}_i - b}{y_i \|\boldsymbol{w}\|}$$

- Let us look at what would we infer for the different values of D?



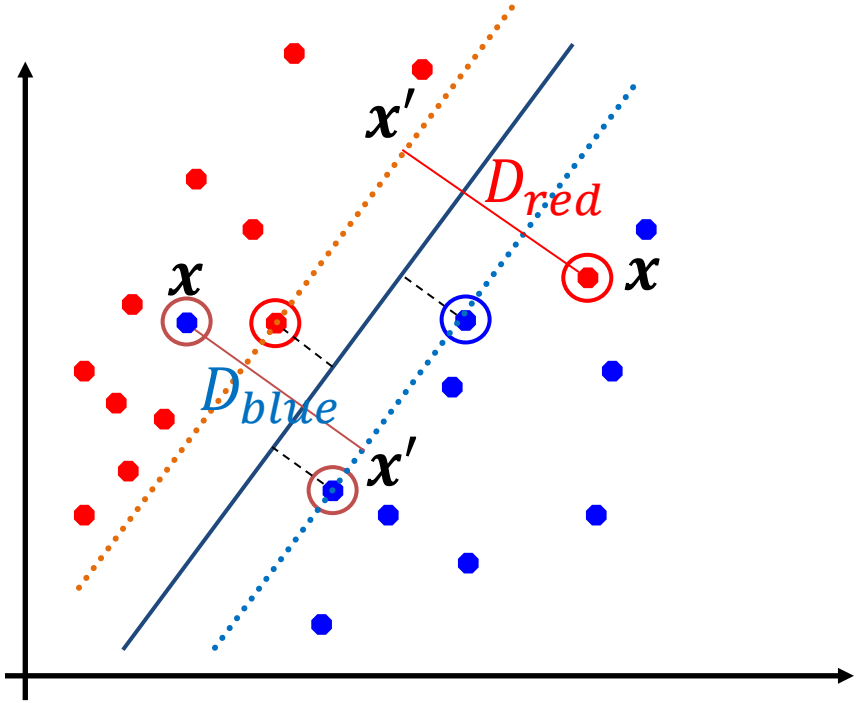
© Copyright National University of Singapore. All Rights Reserved.

Soft-Margin SVM: Slack Variables

- How ε differs from (and relates to) the distance D:
- If $D = 0$, then $\boldsymbol{w}^T\boldsymbol{x}_i + b = y_i$ or $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) > 1$: correctly classified on or outside of the margin.
- If $0 < D \leq 1$, then $0 < \frac{y_i - \boldsymbol{w}^T\boldsymbol{x}_i - b}{y_i\|\boldsymbol{w}\|} \leq 1$:

$$\begin{cases} -1 < \boldsymbol{w}^T\boldsymbol{x}_i + b \leq \|\boldsymbol{w}\| - 1 & \text{if } y_i = -1 \\ 1 - \|\boldsymbol{w}\| \leq \boldsymbol{w}^T\boldsymbol{x}_i + b < 1 & \text{if } y_i = 1 \end{cases}$$

  Therefore: $1 - \|\boldsymbol{w}\| \leq y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) < 1$: correctly classified inside the margin or misclassified.
- If $D > 1$, then $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) < 1 - \|\boldsymbol{w}\|$: correctly classified inside the margin or misclassified.



© Copyright National University of Singapore. All Rights Reserved.

Soft-Margin SVM: Slack Variables

- If $\varepsilon_i = 0$, then $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1$: correctly classified on or outside of the margin.
- If $0 < \varepsilon_i \leq 1$, then $0 \leq y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) < 1$: correctly classified inside the margin.
- If $\varepsilon_i > 1$, then $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) < 0$: misclassified on the wrong side.

- What is the relation between ε and D?
- $\varepsilon_i = D\|w\|$

- If $D = 0$, then $\boldsymbol{w}^T\boldsymbol{x}_i + b = y_i$ or $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) > 1$: correctly classified on or outside of the margin.
- If $0 < D \leq 1$, then $0 < \frac{y_i - \boldsymbol{w}^T\boldsymbol{x}_i - b}{y_i\|\boldsymbol{w}\|} \leq 1$:

$$\begin{cases} -1 < \boldsymbol{w}^T\boldsymbol{x}_i + b \leq \|\boldsymbol{w}\| - 1 & \text{if } y_i = -1 \\ 1 - \|\boldsymbol{w}\| \leq \boldsymbol{w}^T\boldsymbol{x}_i + b < 1 & \text{if } y_i = 1 \end{cases}$$

  Therefore: $1 - \|\boldsymbol{w}\| \leq y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) < 1$: correctly classified inside the margin or misclassified.
- If $D > 1$, then $y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) < 1 - \|\boldsymbol{w}\|$: correctly classified inside the margin or misclassified.

© Copyright National University of Singapore. All Rights Reserved.

Soft-Margin SVM: Primal Form

- How to formulate the Soft-Margin SVM Primal Form?
- Key: how to incorporate the slack variable into the primal form to allow tolerance of "some misclassification"?
- Understand: Slack ε_i is literally *how much the point violates the margin requirement*.
  - If larger than 1 – strictly violation, the larger the more serious violation;
  - Within 1 (positive) – slightly violation.
- We thus formulate the Primal form of Soft-Margin SVM as:

$$\min_{\boldsymbol{w},b,\boldsymbol{\varepsilon}} \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + C\sum_i \varepsilon_i\,, \qquad \text{s.t. } y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1 - \varepsilon_i,\ \varepsilon_i \geq 0, \forall i$$

- What is C? How would C affect the formation of Soft-Margin SVM?

© Copyright National University of Singapore. All Rights Reserved.

Soft-Margin SVM: Primal Form

- Soft-Margin SVM:

$$\min_{\boldsymbol{w},b,\boldsymbol{\varepsilon}} \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + C\sum_i \varepsilon_i\,, \qquad \text{s.t. } y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1 - \varepsilon_i,\ \varepsilon_i \geq 0, \forall i$$

- Hard-Margin SVM:

$$\min_{w,b} \frac{1}{2}w^T w\,, \text{s.t.}\, y_i(w^T x_i + b) \geq 1, \forall i$$

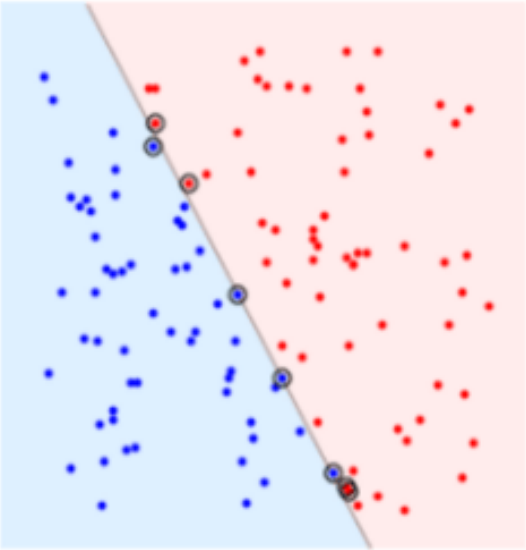
- C: the trade-off between the margin maximization and misclassification error minimization – "There is no free lunch"

C	ε_i	Tolerance for misclassification	Margin	Variance	Bias
Larger	Smaller	Less	Smaller	Higher	Lower
Smaller	Larger	More	Larger	Lower	Higher

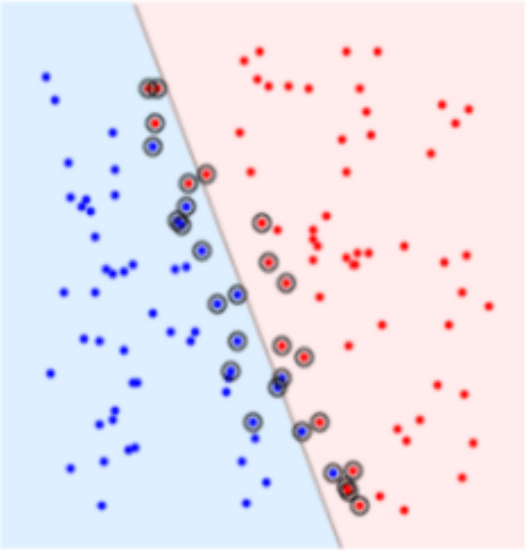
© Copyright National University of Singapore. All Rights Reserved.

Soft-Margin SVM: Hyperparameter C

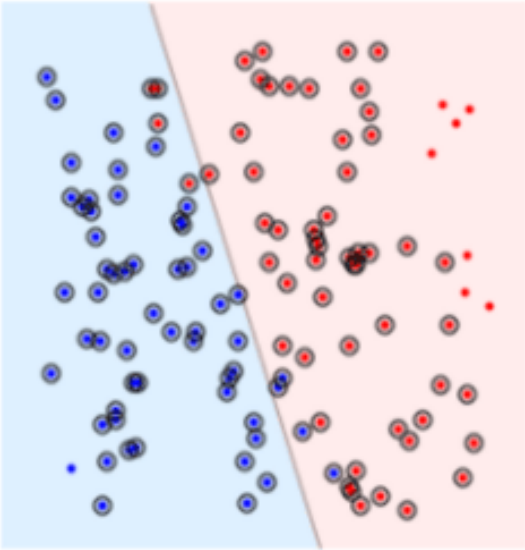
$C = 1000$



$C = 10$



$C = 0.1$



- The larger the C value is, the smaller the margin is.
- How to set the C value if we are to equivalent the soft-margin SVM to a hard-margin SVM?
- $C = 1000$, allow for very small margin as if it is a hard-margin SVM.

© Copyright National University of Singapore. All Rights Reserved.

From Constraint to Loss Function

- Recall the primal form of Soft-Margin SVM:

$$\min_{\boldsymbol{w},b,\boldsymbol{\varepsilon}} \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + C\sum_i \varepsilon_i\,, \qquad \text{s.t. } y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1 - \varepsilon_i,\ \varepsilon_i \geq 0, \forall i$$

- There are essentially two components – an objective (loss) + a constraint.
  - Can we unify these two into one single objective – i.e., can we transform the constraint into a loss function?
  - For simplicity, define $f(x_i) = \boldsymbol{w}^T\boldsymbol{x}_i + b$.
- Look back at the soft margin constraints: $y_i\big(f(x_i)\big) \geq 1 - \varepsilon_i,\ \varepsilon_i \geq 0$:
  - The minimal ε_i needed to satisfy the constraint can be found by:

$$\varepsilon_i = \max\big(0, 1 - y_i f(x_i)\big)$$

  - Let us transform this into a loss function.

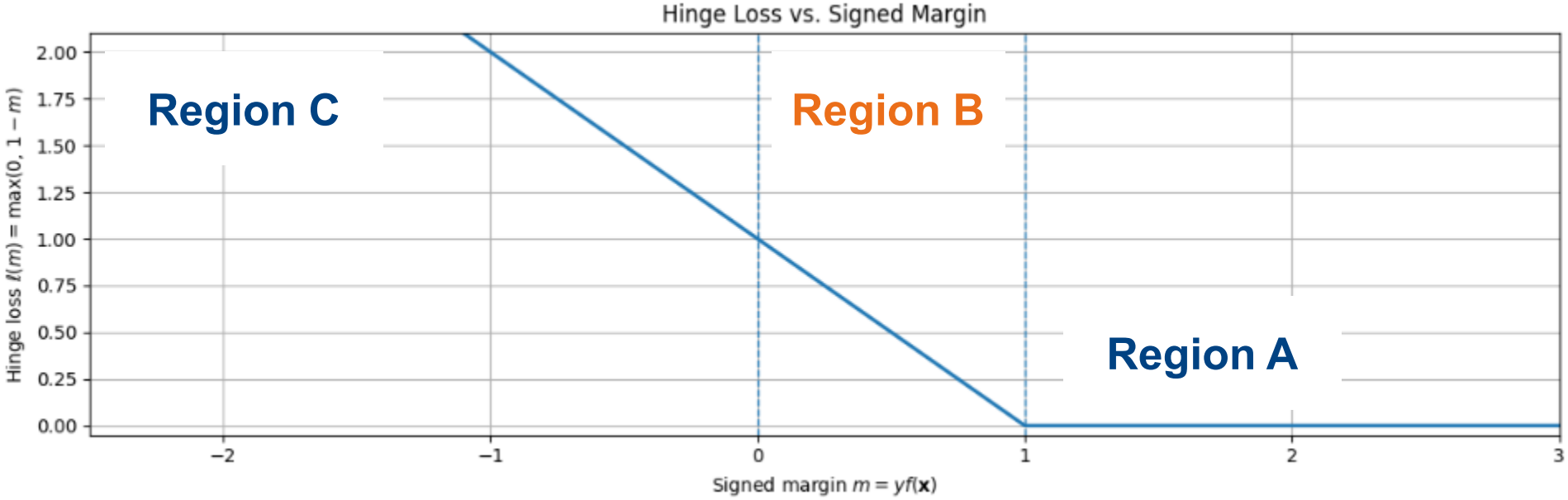
© Copyright National University of Singapore. All Rights Reserved.

Soft-Margin SVM Constraint to Hinge Loss

- Define hinge loss:

$$l_h = \max\left(0, 1 - y_i(w^T x_i + b)\right)$$

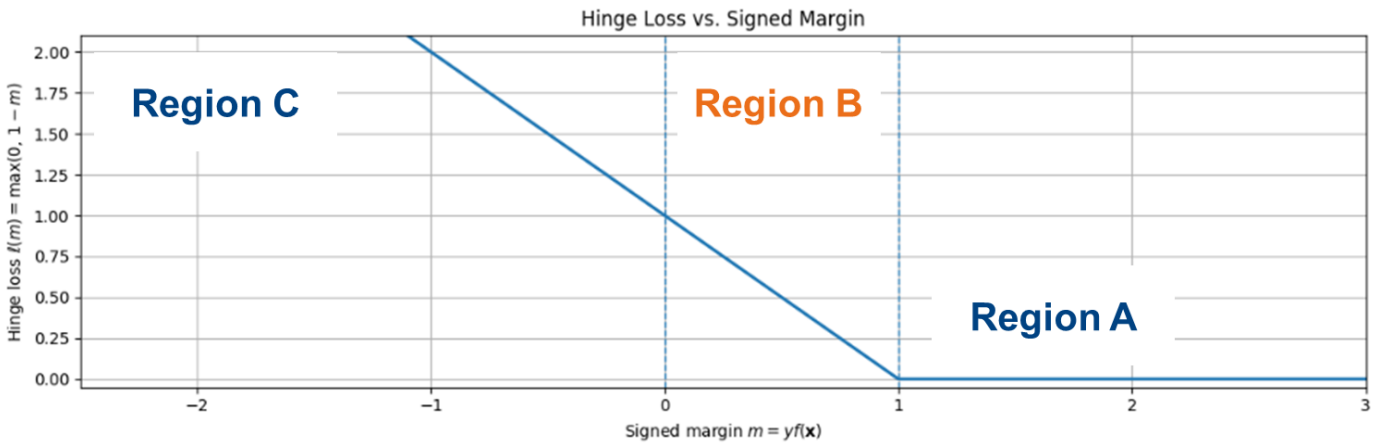
- What does it penalize?
  - Plot a diagram of l_h vs. $m = y\left(f(x)\right)$, denote m as the "signed margin".



© Copyright National University of Singapore. All Rights Reserved.

Soft-Margin SVM Constraint to Hinge Loss

- Region A: $m \geq 1$
  - Correct and outside of margin
  - $l_h = 0$: no penalty.
- Region B: $0 < m < 1$
  - Correct but inside margin
  - $l_h = 1 - m$: linear penalty.
- Region C: $m \leq 0$
  - Misclassified
  - $l_h \geq 1$: larger penalty.



- Hinge loss only cares about points that are either misclassified or too close to the boundary.
- If a point is already confidently classified with margin ≥ 1, the model gains nothing by pushing it further away, so the loss is zero

© Copyright National University of Singapore. All Rights Reserved.

Soft-Margin SVM: Primal with Hinge Loss

$$\min_{\boldsymbol{w},b,\boldsymbol{\varepsilon}} \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + C\sum_i \varepsilon_i\,, \qquad \text{s.t. } y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1 - \varepsilon_i,\ \varepsilon_i \geq 0, \forall i$$

- Substituting the slack variable and the constraint with the hinge loss:

$$\min_{\boldsymbol{w},b,\boldsymbol{\varepsilon}} \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + C\sum_i \max\big(0, 1 - y_i(w^T x_i + b)\big)$$

- Interpretations:
  - $\frac{1}{2}\boldsymbol{w}^T\boldsymbol{w}$: encourages large margin (regularization);
  - $\sum_i l_h$: penalizes margin violations;
  - Large C: prioritize fewer violations (risk overfit);
  - Small C: prioritize wider margin (more regularization).

© Copyright National University of Singapore. All Rights Reserved.

Soft-Margin SVM Dual from KKT Conditions

- Similar to Hard-Margin SVM, we can transform the Soft-Margin primal form into the dual form through KKT conditions.
- Step 1: Define the Lagrangian function:

$$f(\boldsymbol{w}, b, \varepsilon) = \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + C\sum_i \varepsilon_i\,,$$
$$g_i(\boldsymbol{w}, b, \varepsilon) = y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) + \varepsilon_i - 1 \geq 0,$$
$$h_i(\boldsymbol{w}, b, \varepsilon) = \varepsilon_i \geq 0$$

  - Note the difference: we now view the existence of slack variable as a “equality” condition – constraint of variable existence.
- The Lagrangian function is thus:

$$\mathcal{L}(\boldsymbol{w}, b, \boldsymbol{\varepsilon}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = f(\boldsymbol{w}, b, \varepsilon) - \sum_i \lambda_i g_i(\boldsymbol{w}, b, \varepsilon) + \mu_i h_i(\boldsymbol{w}, b, \varepsilon), \lambda_i, \mu_i \geq 0, \forall i$$

© Copyright National University of Singapore. All Rights Reserved.

Soft-Margin SVM Dual from KKT Conditions

- Step 2: Four KKT conditions for optimality:
- A. Stationarity:

$$\begin{cases} \dfrac{\partial \mathcal{L}}{\partial w^*} = 0 \ \rightarrow w^* - \sum_i \lambda_i y_i x_i = 0 \ \rightarrow w^* = \sum_i \lambda_i y_i x_i \\ \dfrac{\partial \mathcal{L}}{\partial b^*} = 0 \ \rightarrow \sum_i \lambda_i y_i = 0 \\ \dfrac{\partial \mathcal{L}}{\partial \epsilon_i^*} = 0 \ \rightarrow C - \lambda_i - \mu_i = 0 \ \rightarrow C = \lambda_i + \mu_i \end{cases}$$

- B. Primal Feasibility:

$$g_i(\boldsymbol{w}, b, \boldsymbol{\varepsilon}) = y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) + \varepsilon_i - 1 \geq 0, \qquad h_i(\boldsymbol{w}, b, \boldsymbol{\varepsilon}) = \ \varepsilon_i \geq 0$$

Soft-Margin SVM Dual from KKT Conditions

- Step 2: Four KKT conditions for optimality:
- C. Dual Feasibility:

$$\lambda_i, \mu_i \geq 0, \forall i$$

- D. Complementary Slackness:

$$\lambda_i g_i(\boldsymbol{w}, b, \boldsymbol{\varepsilon}) = 0, \qquad \mu_i h_i(\boldsymbol{w}, b, \boldsymbol{\varepsilon}) = 0, \forall i$$

- More specifically:

$$\begin{cases} \text{if } g_i(\boldsymbol{w}, b, \boldsymbol{\varepsilon}) > 0 \text{ (inactive)}, & \lambda_i = 0 \\ \text{if } g_i(\boldsymbol{w}, b, \boldsymbol{\varepsilon}) = 0 \text{ (active)}, & \lambda_i > 0 \end{cases}$$

$$\begin{cases} \text{if } h_i(\boldsymbol{w}, b, \boldsymbol{\varepsilon}) > 0 \text{(inactive)}, & \mu_i = 0 \\ \text{if } h_i(\boldsymbol{w}, b, \boldsymbol{\varepsilon}) = 0 \text{(active)}, & \mu_i > 0 \end{cases}$$

© Copyright National University of Singapore. All Rights Reserved.

Soft-Margin SVM Dual from KKT Conditions

- Interpretation of Complementary Slackness:
- (1) When $\lambda_i = 0, \mu_i = C > 0$:

$$\begin{cases} g_i(\boldsymbol{w}, b, \boldsymbol{\varepsilon}) = y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) + \varepsilon_i - 1 > 0 \\ h_i(\boldsymbol{w}, b, \boldsymbol{\varepsilon}) = \varepsilon_i = 0 \end{cases}$$

$$y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) > 1$$

  - Case A: correctly classified and outside the margin.
- (2) When $\lambda_i > 0$, $y_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) = 1 - \varepsilon_i$:
  - Case 1: $\varepsilon_i = 0, \mu_i > 0$ s – on the margin boundaries.
  - Case 2: $0 < \varepsilon_i \leq 1, \mu_i = 0$ s – correctly classified, inside the margin.
  - Case 3: $\varepsilon_i > 1, \mu_i = 0$ s – misclassified.
- Misclassified or margin-violating points are controlled by λ_i, not μ_i.

© Copyright National University of Singapore. All Rights Reserved.

Deriving the Soft-Margin SVM Dual Form

- Now, let us formally derive the SVM dual form:

$$\min_{\boldsymbol{w},b,\boldsymbol{\varepsilon}} \mathcal{L}(\boldsymbol{w}, b, \boldsymbol{\varepsilon}, \boldsymbol{\lambda}, \boldsymbol{\mu})$$

$$= \min_{w,b,\varepsilon} f(\boldsymbol{w}, b, \boldsymbol{\varepsilon}) - \sum_i \lambda_i g_i(\boldsymbol{w}, b, \boldsymbol{\varepsilon}) + \mu_i h_i(\boldsymbol{w}, b, \boldsymbol{\varepsilon}), \text{where } \lambda_i, \mu_i \geq 0, \forall i$$

$$= \min_{w,b,\varepsilon} \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + C\sum_i \varepsilon_i - \sum_i \lambda_i(y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) + \varepsilon_i - 1) + \mu_i\varepsilon_i$$

$$= \min_{w,b,\varepsilon} \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + C\sum_i \varepsilon_i - \sum_i \lambda_i y_i \boldsymbol{w}^T\boldsymbol{x}_i - \sum_i \lambda_i y_i b - \sum_i \lambda_i\varepsilon_i + \sum_i \lambda_i - \sum_i \mu_i\varepsilon_i$$

$$= \frac{1}{2}\boldsymbol{w}^{*T}\boldsymbol{w}^* - \boldsymbol{w}^{*T}\boldsymbol{w}^* + \sum_i \lambda_i = -\frac{1}{2}\sum_i \lambda_i y_i \boldsymbol{x}_i^T \sum_i \lambda_i y_i \boldsymbol{x}_i + \sum_i \lambda_i$$

© Copyright National University of Singapore. All Rights Reserved.

Deriving the Soft-Margin SVM Dual Form

- Now, let us formally derive the SVM dual form:

$$\min_{\boldsymbol{w},b,\boldsymbol{\varepsilon}} \mathcal{L}(\boldsymbol{w}, b, \boldsymbol{\varepsilon}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = -\frac{1}{2}\sum_i \lambda_i y_i \boldsymbol{x}_i^T \sum_i \lambda_i y_i \boldsymbol{x}_i + \sum_i \lambda_i = -\frac{1}{2}\sum_{i,j} \lambda_i \lambda_j y_i y_j \boldsymbol{x}_i^T \boldsymbol{x}_j + \sum_i \lambda_i$$

- Therefore, the dual form of soft-margin SVM is the optimization problem:

$$\max_{\boldsymbol{\lambda}} -\frac{1}{2}\sum_{i,j} \lambda_i \lambda_j y_i y_j \boldsymbol{x}_i^T \boldsymbol{x}_j + \sum_i \lambda_i \text{, s.t. } 0 \le \lambda_i \le C, \ \sum_i \lambda_i y_i = 0, \forall i$$

- Note in our derivation we define:

$$\begin{cases} f(\boldsymbol{w}, b, \boldsymbol{\varepsilon}) = \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + C\sum_i \varepsilon_i, \\ g_i(\boldsymbol{w}, b, \boldsymbol{\varepsilon}) = y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) + \varepsilon_i - 1 \ge 0, \\ h_i(\boldsymbol{w}, b, \boldsymbol{\varepsilon}) = \varepsilon_i \ge 0 \end{cases}$$

© Copyright National University of Singapore. All Rights Reserved.

Soft-Margin SVM Dual Form: Summary

- We summarize the optimal results for weights w:

$$\boldsymbol{w}^* = \sum_i \lambda_i y_i \boldsymbol{x}_i \,, \sum_i \lambda_i y_i = 0, C = \lambda_i + \mu_i$$

- And subsequently the optimal bias b:

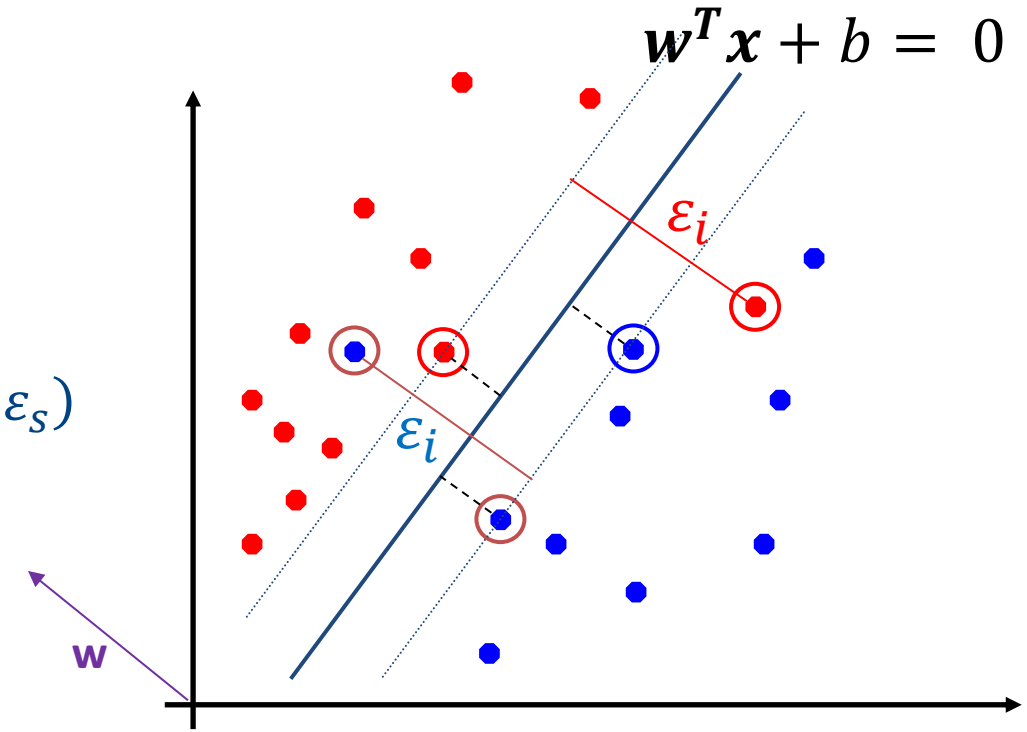
$$\boldsymbol{w}^{*T}\boldsymbol{x} + b^* = \sum_i \lambda_i y_i \boldsymbol{x}_i^T \boldsymbol{x} + b^* = 0$$

for any support vector $\boldsymbol{x}_s$ $(\lambda_i > 0)$,

$$y_s(\boldsymbol{w}^{*T}\boldsymbol{x}_s + b^*) = 1 - \varepsilon_s \;\rightarrow y_s^2(\boldsymbol{w}^{*T}\boldsymbol{x}_s + b^*) = y_s(1 - \varepsilon_s)$$

$$b^* = y_s(1 - \varepsilon_s) - \sum_i \lambda_i y_i \boldsymbol{x}_i^T \boldsymbol{x}_s, \text{or}$$

$$b^* = \frac{1}{N_s}\sum_s \left(y_s(1 - \varepsilon_s) - \sum_i \lambda_i y_i \boldsymbol{x}_i^T \boldsymbol{x}_s\right)$$



© Copyright National University of Singapore. All Rights Reserved.

Quick Review Question

For Soft-Margin SVM with the following objective function:

$$\min_{\boldsymbol{w},b,\boldsymbol{\varepsilon}} \frac{1}{2}\boldsymbol{w}^T\boldsymbol{w} + C\sum_i \varepsilon_i$$

$$\text{s.t. } y_i(\boldsymbol{w}^T\boldsymbol{x}_i + b) \geq 1 - \varepsilon_i, \varepsilon_i \geq 0, \forall i$$

or

$$\max_{\boldsymbol{\lambda}} -\frac{1}{2}\sum_{i,j} \lambda_i\lambda_j y_i y_j \boldsymbol{x}_i^T\boldsymbol{x}_j + \sum_i \lambda_i$$

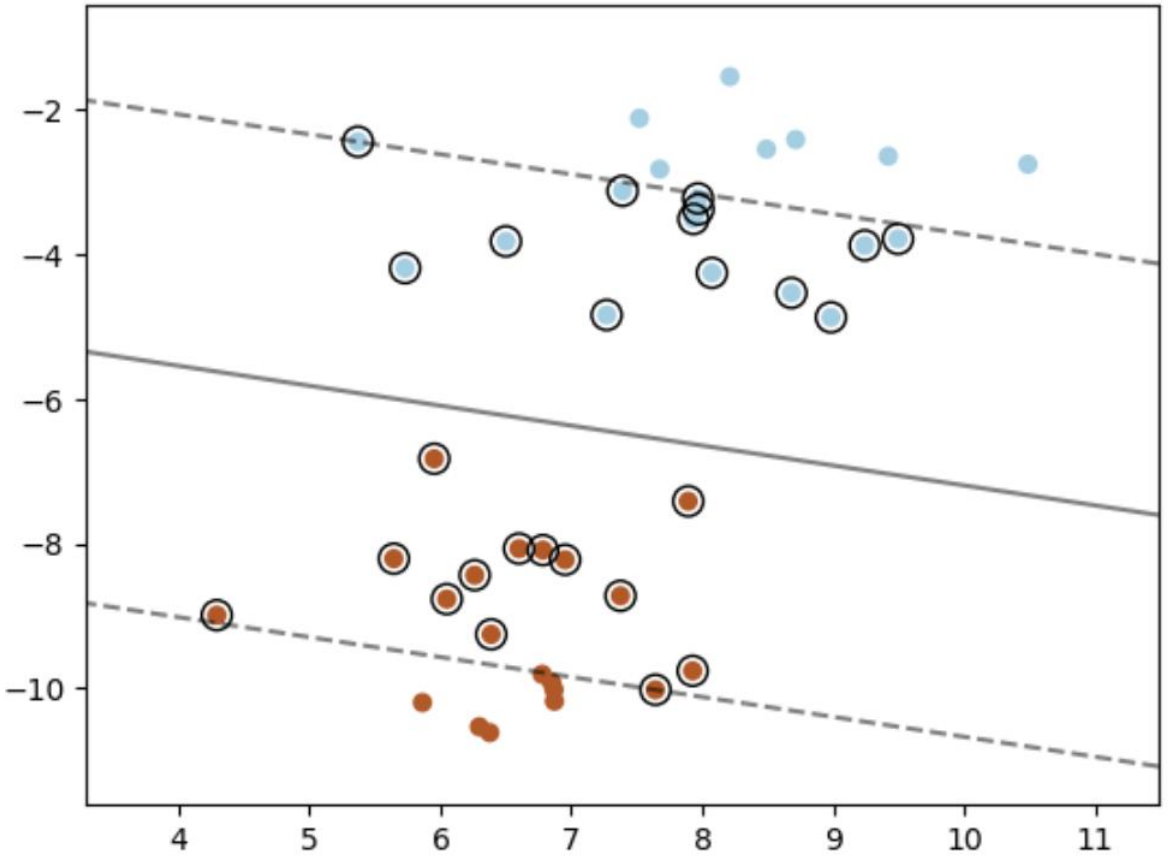
$$\text{s.t. } 0 \leq \lambda_i \leq C, \sum_i \lambda_i y_i = 0, \forall i$$

What are the support vectors?

A. Data points that lie exactly on the margin boundaries.
B. Data points that have $\lambda_i > 0$.
C. Data points that determine the decision boundary
D. Data points inside the margin
E. Data points that are misclassified
F. Data points that have $\varepsilon_i \geq 0$

© Copyright National University of Singapore. All Rights Reserved.

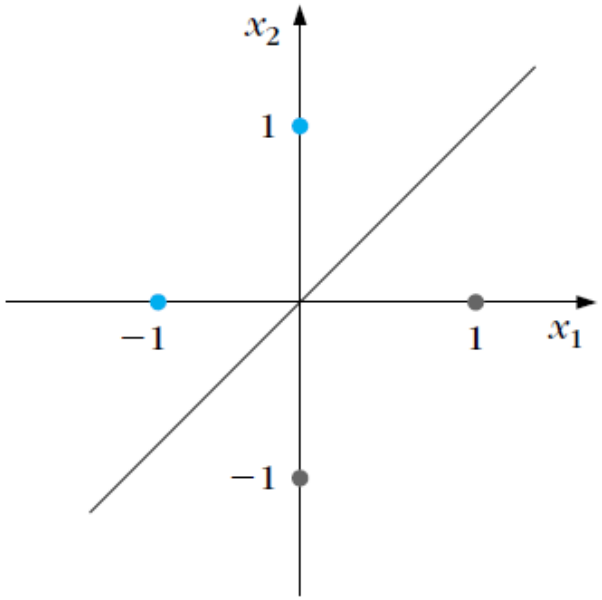
Demo: Soft-Margin SVM with Scikit-Learn



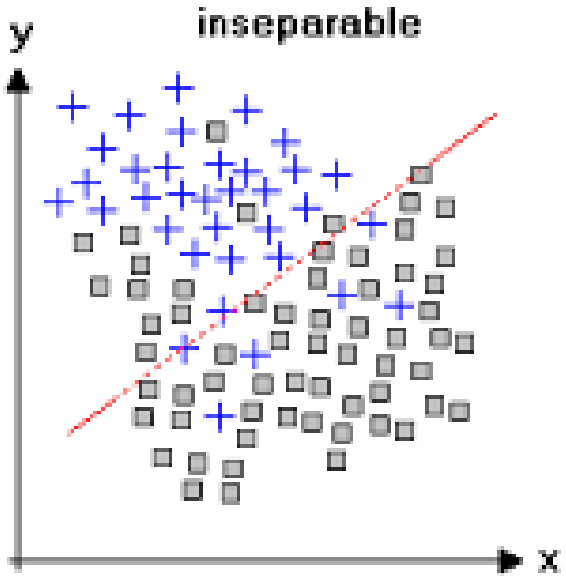
© Copyright National University of Singapore. All Rights Reserved.

SVM: Linearly Inseparable Data

- Recall the motivation for introducing Soft-Margin SVM: the class are not linearly separable.
- With slack variables, soft-margin SVMs extends linear classifier to more possible scenarios.
- What are the limitations of soft-margin SVMs?



- Still, there is a strong assumption:
  The number of data that falls into the margin or are misclassified must be **significantly few** w.r.t. total data.
- Cases where the ratio of misclassification / in-margin data is still significant even with optimal linear classifiers?

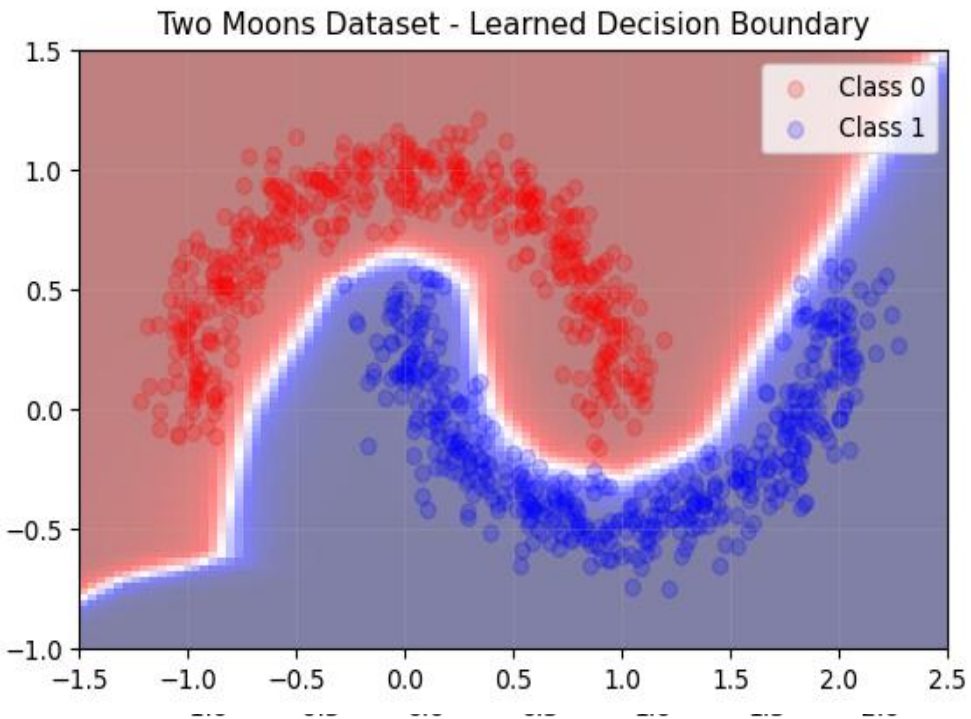


© Copyright National University of Singapore. All Rights Reserved.

XOR Problem and Two Moon Dataset

x_1	x_2	XOR	Class
0	0	0	B
0	1	1	A
1	0	1	A
1	1	0	B

- Two typical examples: XOR and Two Moon.
- Example 1: XOR
  - How-ever we try to create a linear classifier, at least 1 out of 4 misclassifications.
  - Misclassification rate ≥ 25%!
- Example 2: Two Moon
  - Misclassification rate for an optimal linear classifier is also ~25%!
  - The actual best classifier should actually be an S-shaped curve.
- Linear classifiers don't seem to fit these examples. ☹

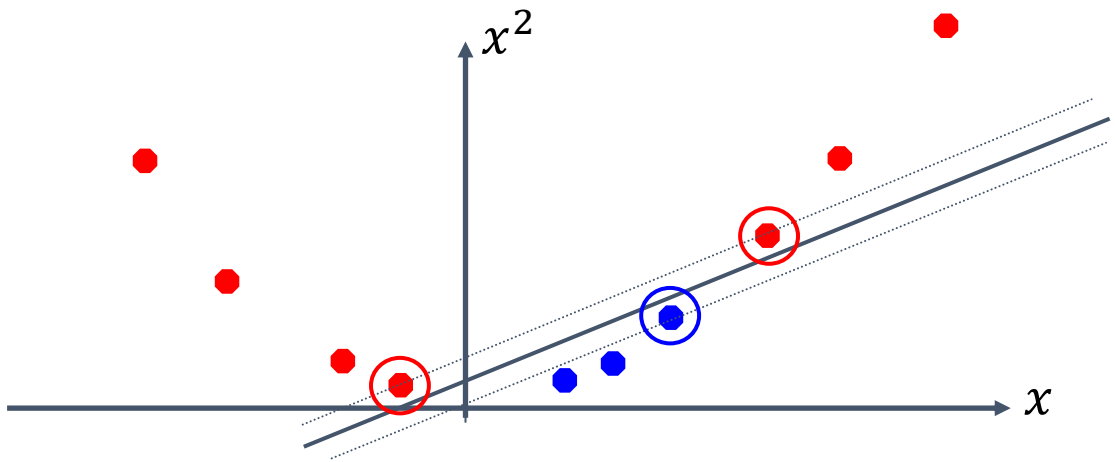


© Copyright National University of Singapore. All Rights Reserved.

Kernel SVM: Changing Data Representation

- If we cannot classify linearly in the current space, can we change the data representation to another space so that it is linearly separable?
- Here's a linearly non-separable dataset in 1-D space

- We can map data x from 1-D to 2-D by $x \rightarrow (x, x^2)$

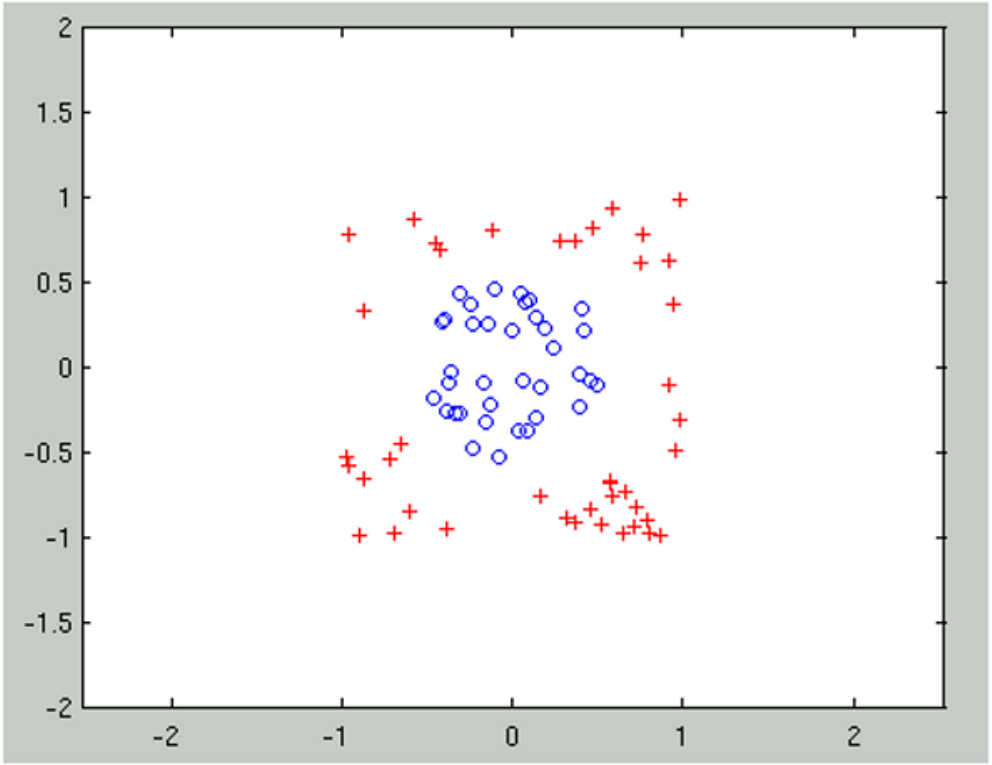


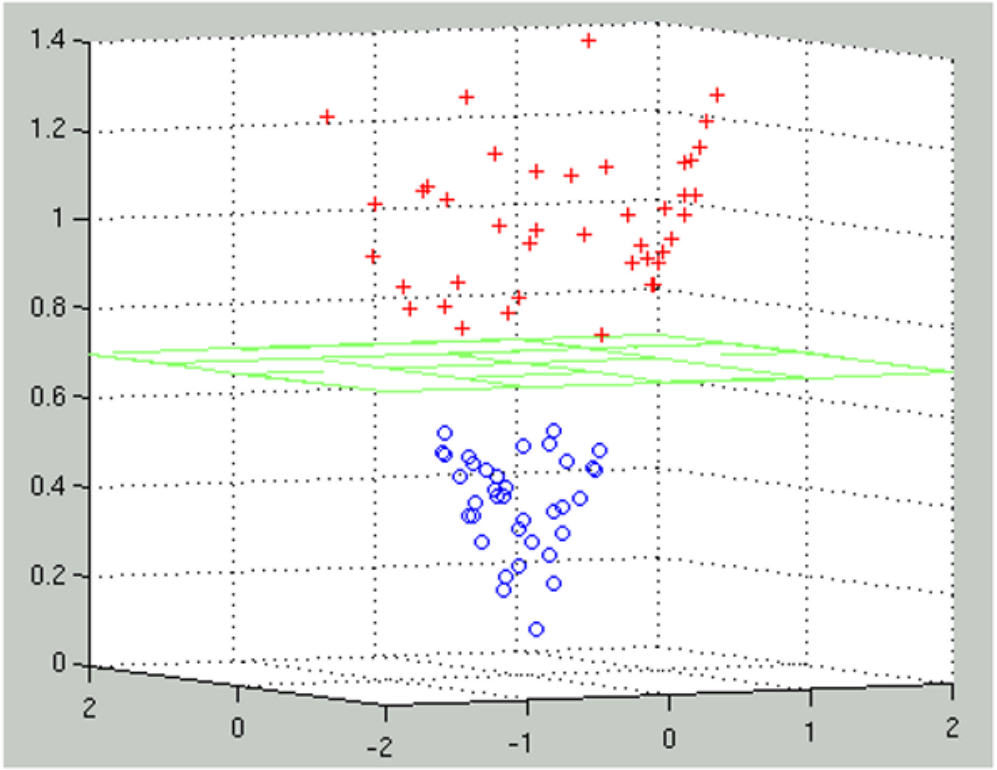
- With this projection, now the data is linearly separable in the new space.
- We can run SVM in the new space
- The linear decision boundary in the new space corresponds to a non-linear decision boundary in the old space

© Copyright National University of Singapore. All Rights Reserved.

Kernel SVM: Changing Data Representation

- Another example: $(x_1, x_2) \Rightarrow \left(x_1, x_2, \sqrt{x_1^2 + x_2^2}\right)$





© Copyright National University of Singapore. All Rights Reserved.

Kernel SVM: Separable in Feature Space

- General idea: map the original feature space $x = \left(x^1, x^2, \dots, x^d\right)^T$ to some higher dimensional feature space $\phi(x)$ where the data is separable.
- The key: apply a feature mapping $\phi: \mathbb{R}^d \to \mathbb{R}^D, d < (\ll) D$
  - Essentially, the same SVM(s) we discuss, but in a higher dimensional space.
- Decision boundary:

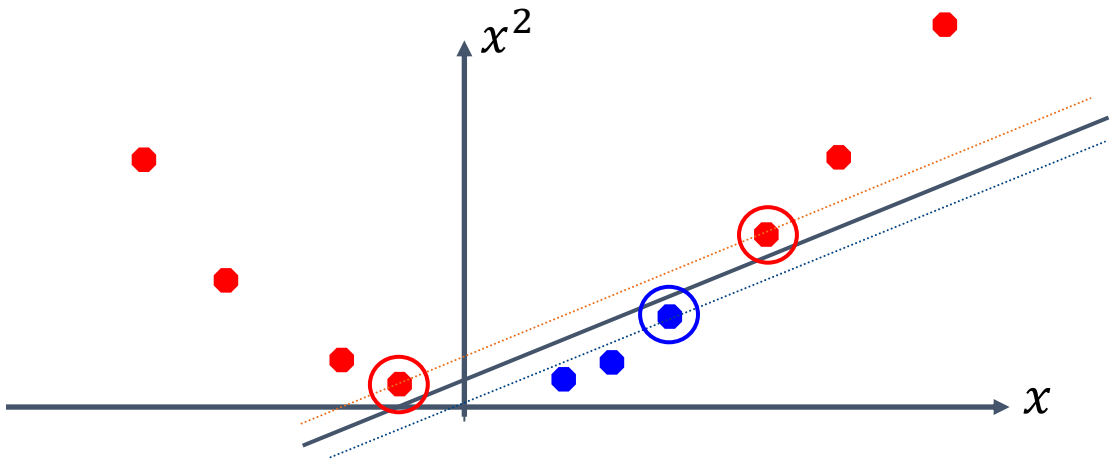
$$w^T \phi(x) + b = 0$$

- Plus plane:

$$w^T \phi(x) + b = 1$$

- Minus plane

$$w^T \phi(x) + b = -1$$



© Copyright National University of Singapore. All Rights Reserved.

Kernel SVM: Formula with Mapping Function

- The optimization formulation after mapping ($x \to \phi(x)$):
  - For Hard-Margin (primal and dual forms):

$$\min_{w,b} \frac{1}{2} w^T w \,, \text{s.t.}\, y_i(w^T \phi(x_i) + b) \geq 1, \forall i$$

$$\max_{\boldsymbol{\lambda}} -\frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j \phi(x_i)^T \phi(x_j) + \sum_i \lambda_i \,, s.t.\ \lambda_i \geq 0,\ \sum_i \lambda_i y_i = 0, \forall i$$

  - For Soft-Margin (primal and dual forms):

$$\min_{\boldsymbol{w},b,\boldsymbol{\varepsilon}} \frac{1}{2} \boldsymbol{w}^T \boldsymbol{w} + C \sum_i \varepsilon_i \,, \qquad \text{s.t. } y_i(\boldsymbol{w}^T \phi(x_i) + b) \geq 1 - \varepsilon_i,\ \varepsilon_i \geq 0, \forall i$$

$$\max_{\boldsymbol{\lambda}} -\frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j \phi(x_i)^T \phi(x_j) + \sum_i \lambda_i \,, \text{s.t. } 0 \leq \lambda_i \leq C,\ \sum_i \lambda_i y_i = 0, \forall i$$

© Copyright National University of Singapore. All Rights Reserved.

Kernel SVM: Formula with Mapping Function

- Optimal decision boundary after mapping ($x \to \phi(x)$):
  - For Hard-Margin SVM:

$$w^{*T}\phi(x) + b^* = 0, \qquad w^* = \sum_i \lambda_i y_i \phi(x_i)$$

$$b^* = y_s - \sum_i \lambda_i y_i \phi(x_i)^T \phi(x_s) = \frac{1}{N_s}\sum_s \left(y_s - \sum_i \lambda_i y_i \phi(x_i)^T \phi(x_s) \right)$$

  - For Soft-Margin SVM:

$$w^{*T}\phi(x) + b^* = 0, \qquad \boldsymbol{w}^* = \sum_i \lambda_i y_i \phi(x_i), \sum_i \lambda_i y_i = 0, C = \lambda_i + \mu_i$$

$$b^* = y_s(1-\varepsilon_s) - \sum_i \lambda_i y_i \phi(x_i)^T \phi(x_s) = \frac{1}{N_s}\sum_s (y_s(1-\varepsilon_s) - \sum_i \lambda_i y_i \phi(x_i)^T \phi(x_s))$$

© Copyright National University of Singapore. All Rights Reserved.

Kernel SVM: The Kernel Trick

- The formulation is simple: changing all x_i, x_j into $\phi(x_i), \phi(x_j)$
- In short, the original solution relies on the inner product $x_i^T x_j$, the kernel SVM solution relies on the inner product $\phi(x_i)^T \phi(x_j)$

- If I add lots of nonlinear features, what risk increases?
- Probability of overfitting!
- Computation cost (of computing $\phi(\cdot)$ itself and the inner product)!

- How should we cope with the increasing (and mostly unwanted) additional computation cost? The Kernel Trick!

© Copyright National University of Singapore. All Rights Reserved.

Kernel SVM: The Kernel Trick

- The goal: To reduce the computation of the inner product of $\phi(x)$ in a high-d space.

- The Kernel Trick: compute the inner product without explicit $\phi(x)$ – i.e., find a function whose result equals the inner product in the original low-d space.
- Define a kernel function K: $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$
  - E.g.,: $x = (x^1, x^2)$, $\phi(x) = \left((x^1)^2, (x^2)^2, \sqrt{2}(x^1)(x^2), \sqrt{2}(x^1), \sqrt{2}(x^2), 1\right)^T$

$$\phi(x_i)^T \phi(x_j) = \left((x_i^1)^2, (x_i^2)^2, \sqrt{2}(x_i^1)(x_i^2), \sqrt{2}(x_i^1), \sqrt{2}(x_i^2), 1\right) \begin{pmatrix} (x_j^1)^2 \\ (x_j^2)^2 \\ \sqrt{2}(x_j^1)(x_j^2) \\ \sqrt{2}(x_j^1) \\ \sqrt{2}(x_j^2) \\ 1 \end{pmatrix}$$

- Next is magic!

Kernel SVM: The Kernel Trick

$$\phi(x_i)^T\phi(x_j) = \left((x_i^1)^2, (x_i^2)^2, \sqrt{2}(x_i^1)(x_i^2), \sqrt{2}(x_i^1), \sqrt{2}(x_i^2), 1\right)\begin{pmatrix} (x_j^1)^2 \\ (x_j^2)^2 \\ \sqrt{2}(x_j^1)(x_j^2) \\ \sqrt{2}(x_j^1) \\ \sqrt{2}(x_j^2) \\ 1 \end{pmatrix}$$

$$= (x_i^1)^2(x_j^1)^2 + (x_i^2)^2(x_j^2)^2 + 2(x_i^1)(x_j^1)(x_i^2)(x_j^2) + 2(x_j^1)(x_j^1) + 2(x_j^1)(x_j^1) + 1$$

$$= \left((x_i^1)(x_j^1) + (x_i^2)(x_j^2) + 1\right)^2 = (x_i^T x_j + 1)^2 = K(x_i, x_j)$$

- The mapping function maps from $\mathbb{R}^d \to \mathbb{R}^m$ (In Example, $d = 2, m = 6$)
  - Without $K(\cdot) - O(m)$; With $K(\cdot) - O(d)$
- The Kernel Trick with Kernel function saves computation significantly!

© Copyright National University of Singapore. All Rights Reserved.

Kernel SVM: Reformulate with Kernel Function

- Now, let us reformulate the dual form of hard and soft-margin SVMs with their optimized weights and bias with the kernel trick. ☺
- Hard-margin SVM (dual form):

$$\max_{\boldsymbol{\lambda}} -\frac{1}{2}\sum_{i,j}\lambda_i\lambda_j y_i y_j K(x_i, x_j) + \sum_i \lambda_i \,, s.t.\ \lambda_i \geq 0,\ \sum_i \lambda_i y_i = 0, \forall i$$

$$w^{*T}\phi(x) + b^* = \sum_i \lambda_i y_i K(x_i, x) + b^* = 0$$

$$b^* = y_s - \sum_i \lambda_i y_i K(x_i, x_s) = \frac{1}{N_s}\sum_s \left(y_s - \sum_i \lambda_i y_i K(x_i, x_s) \right)$$

© Copyright National University of Singapore. All Rights Reserved.

Kernel SVM: Reformulate with Kernel Function

- Soft-margin SVM (dual form):

$$\max_{\lambda} -\frac{1}{2}\sum_{i,j}\lambda_i\lambda_j y_i y_j K(x_i, x_j) + \sum_i \lambda_i \text{ , s.t. } 0 \le \lambda_i \le C,\ \sum_i \lambda_i y_i = 0, \forall i$$

$$w^{*T}\phi(x) + b^* = \sum_i \lambda_i y_i K(x_i, x) + b^* = 0$$

$$b^* = y_s(1-\varepsilon_s) - \sum_i \lambda_i y_i K(x_i, x_s) = \frac{1}{N_s}\sum_s (y_s(1-\varepsilon_s) - \sum_i \lambda_i y_i K(x_i, x_s))$$

- Note the changes with the kernel trick?
- Since we may not explicit $\phi(x)$, we directly evaluate the ***decision boundary*** with the kernel function K.

© Copyright National University of Singapore. All Rights Reserved.

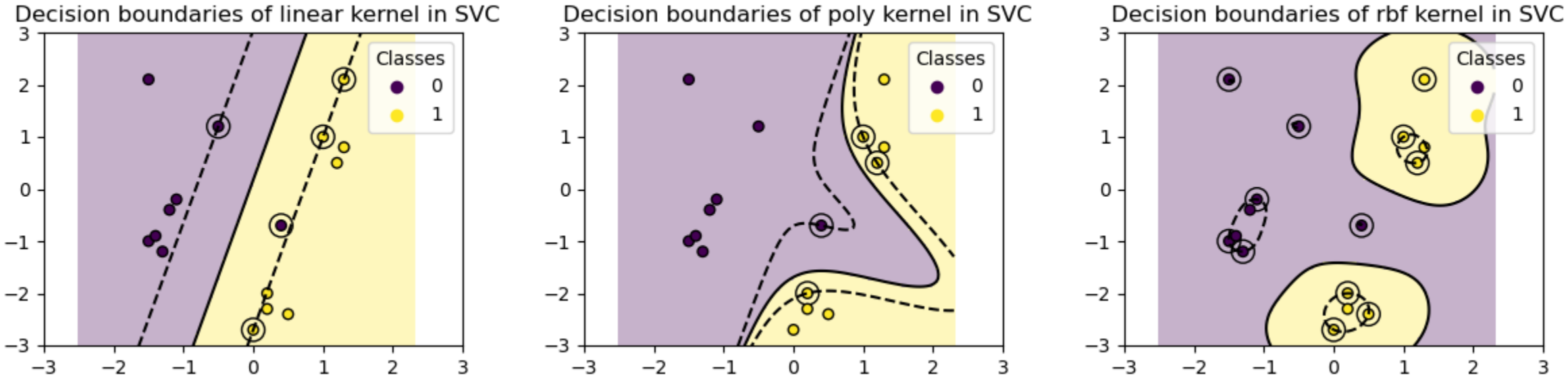
Kernel SVM: What Kernel Functions?

- What kernel function is a good (or at least valid) kernel function?
- Three basic examples first: (for simplicity, we denote as $K(x,z), x, z \in \mathbb{R}^d$)
- 1. Linear kernel (baseline kernel):
  - $K(x,z) = x^T z \Rightarrow \phi(x) = x$
  - Usually used in high-dimensional sparse features.
- 2. Polynomial kernel:
  - $K(x,z) = (\gamma x^T z + C_0)^p \Rightarrow \phi(x) \in \mathbb{R}^m,\ m = \binom{d+p}{d} = C_d^{d+p} = \frac{(d+p)!}{p!d!}$
  - m is usually denoted as the binomial coefficient.
- 3. Gaussian (Radial-Basis Function RBF) kernel (baseline nonlinear kernel):
  - $K(x,z) = \exp(-\gamma\|x-z\|^2) \Rightarrow \phi(x)$ is infinite-dimensional – every point is mapped to a Gaussian function.

© Copyright National University of Singapore. All Rights Reserved.

Kernel SVM: What Kernel Functions?

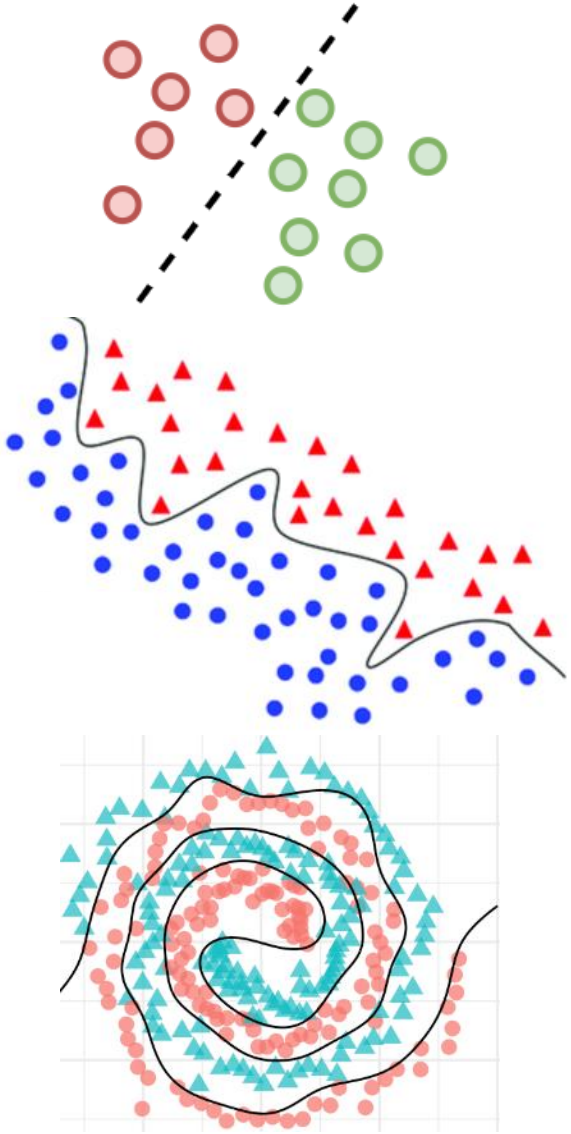
- Quick comparison with the same set of data.
- Implemented with scikit-learn: $\mathrm{sklearn.svm.SVC}$
  - Default kernel is chosen as RBF kernel.
  - Default soft-margin tradeoff (C) is 1.0



© Copyright National University of Singapore. All Rights Reserved.

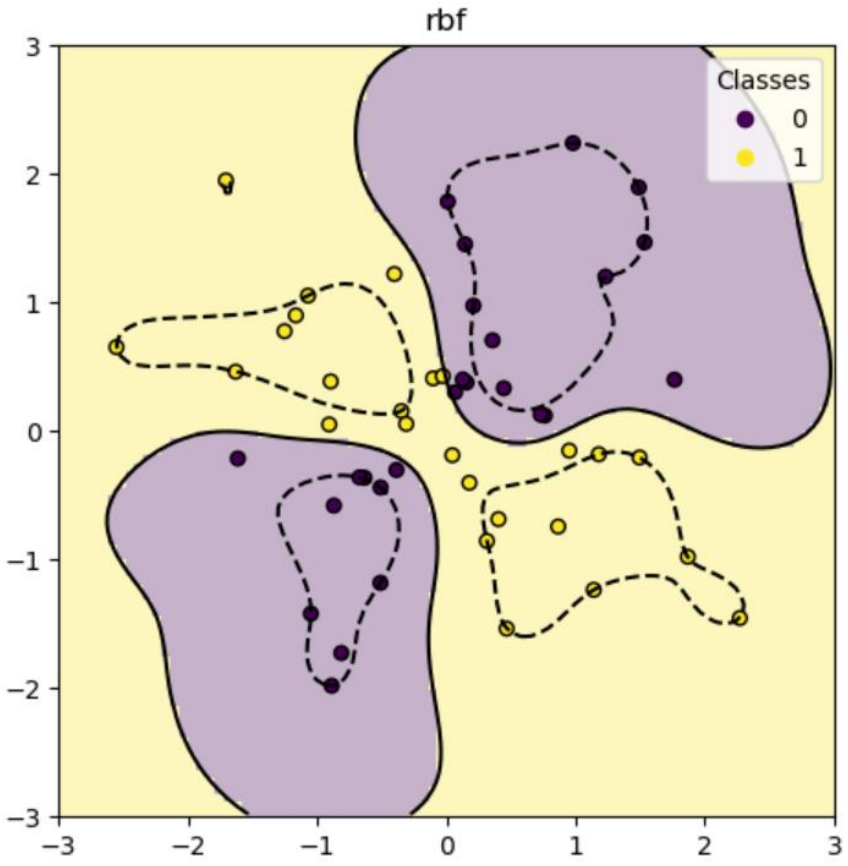
Kernel SVM: Choosing the Right Kernel

- When to use for the three different kernels?
- 1. Linear kernel: $K(x, z) = x^T z$
  - The data is linearly separable.
- 2. Polynomial kernel: $K(x, z) = (\gamma x^T z + C_0)^p$
  - Nonlinear but polynomial-like decision boundary.
  - Capture feature interactions.
  - Low-dimension data.
- 3. Gaussian (RBF) kernel: $K(x, z) = \exp(-\gamma \|x - z\|^2)$
  - Highly complex and nonlinear decision boundary.
  - Unknown underlying feature transformations.
  - Moderate-sized data and moderate-level-dimension data.



© Copyright National University of Singapore. All Rights Reserved.

Demo: Kernels for Kernel SVM with Scikit-Learn



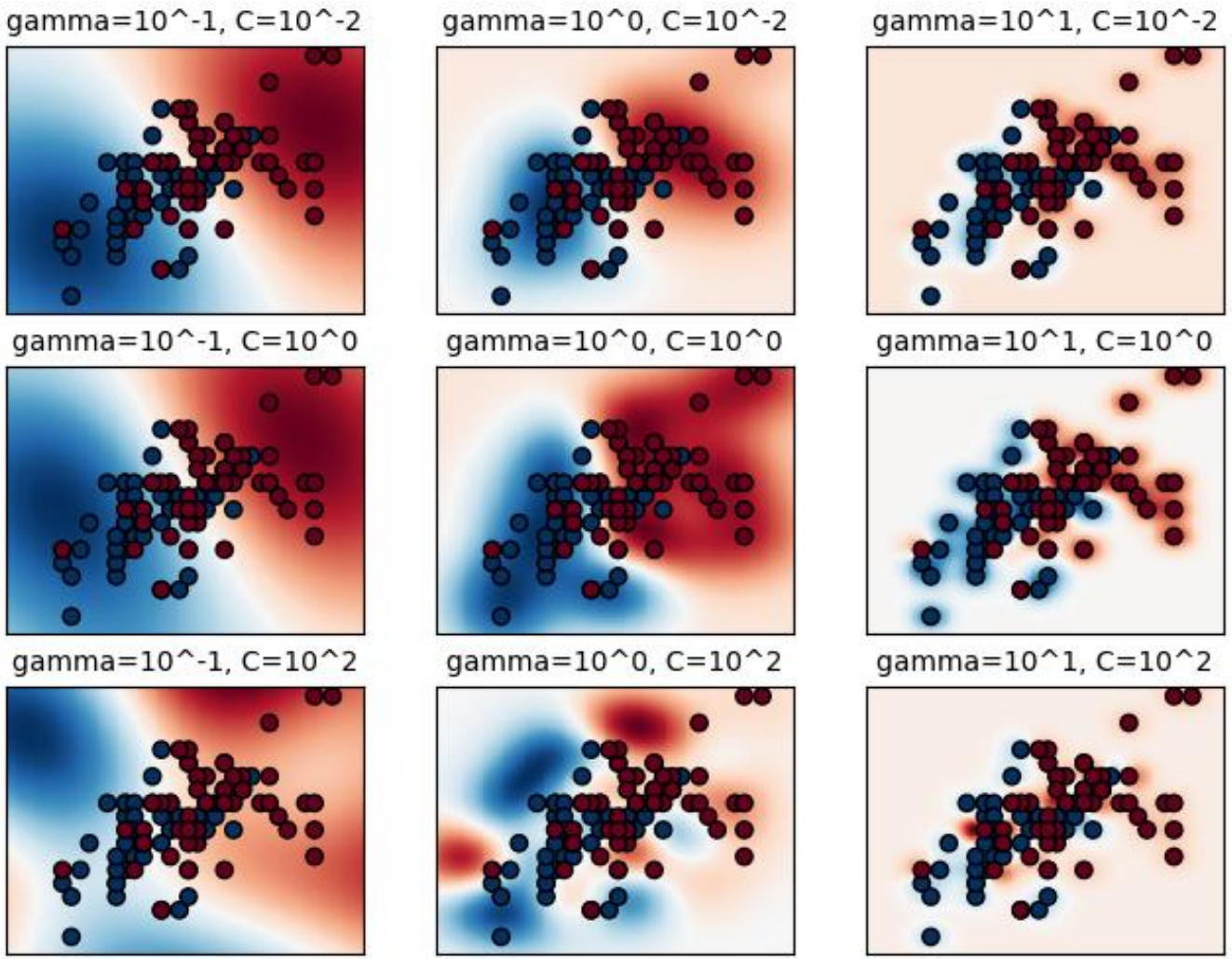
© Copyright National University of Singapore. All Rights Reserved.

Kernel SVM: Impact of Hyperparameters (RBF)

- RBF kernel: $K(x, z) = \exp(-\gamma \|x - z\|^2)$ (Default SVC kernel option).
- Hyperparameter γ: radius of influence
  - Smaller γ: the influence of each data point is far → producing smoother boundary;
  - Larger γ: the influence of each data point is close → more local boundaries.
- Actual implementation would include the soft-margin tradeoff C
  - C tradeoff training errors vs. large margins (the larger C, less regularization)

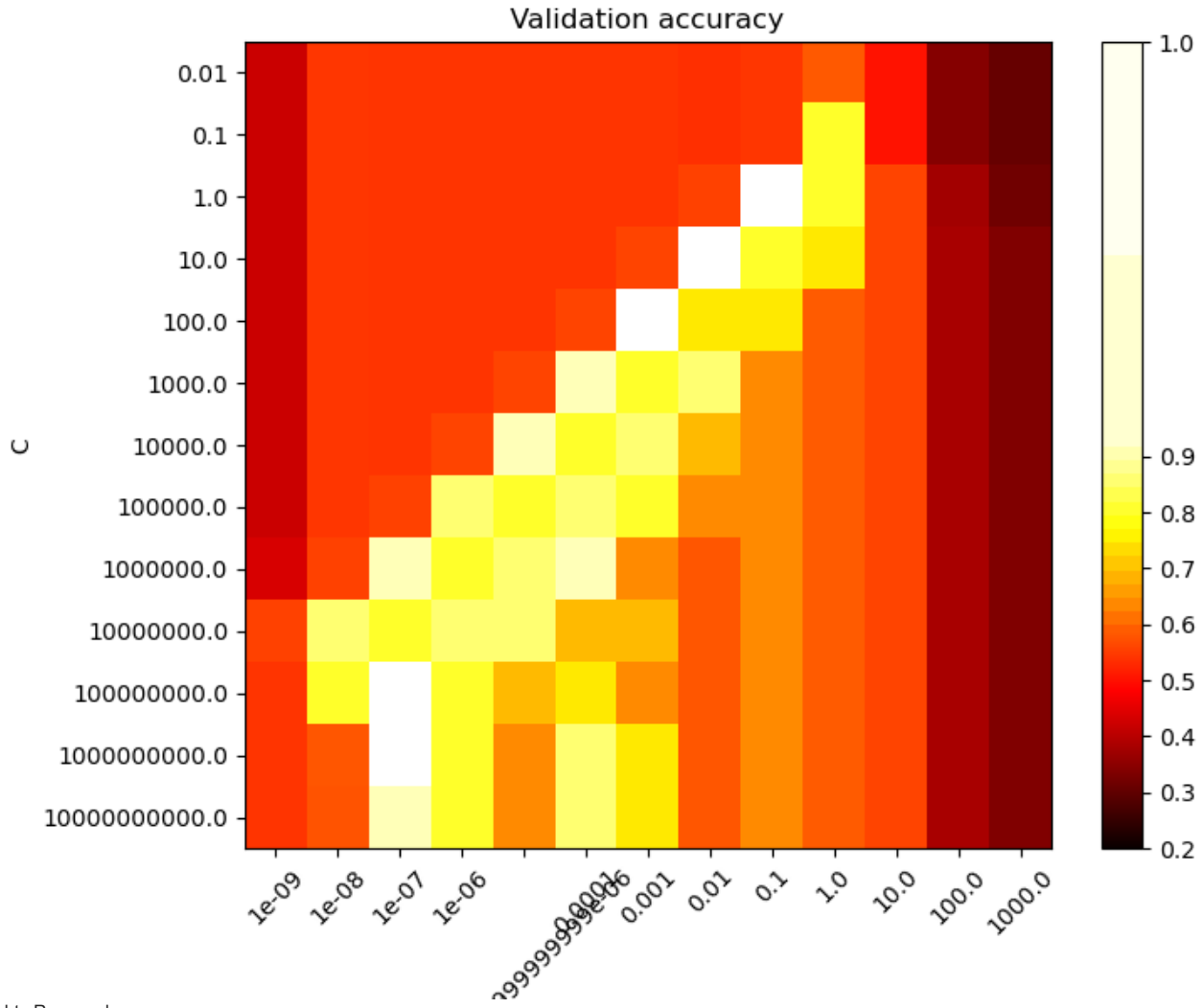
		γ	
		High	Low
C	High	Very flexible + local → **overfit** risk.	Smooth kernel but tries hard to fit → can still be complex.
	Low	Kernel is local but regularization allows mistakes → Less wiggly then above but unstable.	Smooth + strongly regularized → **underfit** risk.

Kernel SVM: Impact of Hyperparameters (RBF)



© Copyright National University of Singapore. All Rights Reserved.

Kernel SVM: Impact of Hyperparameters (RBF)



© Copyright National University of Singapore. All Rights Reserved.

What Functions Can be Kernels?

- Can we always find out a kernel K for a given mapping function ϕ?
- Given a kernel K, can we always find its corresponding ϕ function?

- What functions can be considered as candidate for kernel function K?
- The Mercer's Theorem: for $K(x_i, x_j)$ to be a valid kernel, it must satisfy the Mercer / PSD conditions:
  - i. Symmetry: $K(x_i, x_j) = K(x_j, x_i)$
  - ii. PSD (Positive Semi-Definiteness): for any finite set $\{x_1, x_2, \dots, x_n\}$, the Gram matrix G with $G_{ij} = K(x_i, x_j)$ is Positive Semi-Definite if $v^T G v \geq 0$ for any non-zero vector v.
  - $\boldsymbol{G} = \begin{bmatrix} K(\boldsymbol{x}_1, \boldsymbol{x}_1) & \cdots & K(\boldsymbol{x}_1, \boldsymbol{x}_N) \\ \vdots & \ddots & \vdots \\ K(\boldsymbol{x}_N, \boldsymbol{x}_1) & \cdots & K(\boldsymbol{x}_N, \boldsymbol{x}_N) \end{bmatrix}$

© Copyright National University of Singapore. All Rights Reserved.

The Mercer's Theorem

- Proof of PSD for valid kernels K:

$$\boldsymbol{v}^T \boldsymbol{G} \boldsymbol{v} = \begin{bmatrix} v_1 & \cdots & v_N \end{bmatrix} \begin{bmatrix} K(\boldsymbol{x}_1, \boldsymbol{x}_1) & \cdots & K(\boldsymbol{x}_1, \boldsymbol{x}_N) \\ \vdots & \ddots & \vdots \\ K(\boldsymbol{x}_N, \boldsymbol{x}_1) & \cdots & K(\boldsymbol{x}_N, \boldsymbol{x}_N) \end{bmatrix} \begin{bmatrix} v_1 \\ \vdots \\ v_N \end{bmatrix}$$

$$= \begin{bmatrix} \sum_{i=1}^{N} v_i K(\boldsymbol{x}_i, \boldsymbol{x}_1) & \cdots & \sum_{i=1}^{N} v_i K(\boldsymbol{x}_i, \boldsymbol{x}_N) \end{bmatrix} \begin{bmatrix} v_1 \\ \vdots \\ v_N \end{bmatrix} = \sum_{j=1}^{N} \sum_{i=1}^{N} v_i K(\boldsymbol{x}_i, \boldsymbol{x}_j) v_j$$

- If K is a valid kernel, $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$, we then have:

$$\boldsymbol{v}^T \boldsymbol{G} \boldsymbol{v} = \sum_{j=1}^{N} \sum_{i=1}^{N} v_i K(\boldsymbol{x}_i, \boldsymbol{x}_j) v_j = \sum_{j=1}^{N} \sum_{i=1}^{N} v_i \phi(\boldsymbol{x}_i)^T \phi(\boldsymbol{x}_j) v_j$$

© Copyright National University of Singapore. All Rights Reserved.

The Mercer's Theorem

- Subsequently, because of the Linearity of the inner product:

$$(a\phi(\boldsymbol{x}_i) + b\phi(\boldsymbol{x}_k))^T\phi(\boldsymbol{x}_j) = a\phi(\boldsymbol{x}_i)^T\phi(\boldsymbol{x}_j) + b\phi(\boldsymbol{x}_k)^T$$

$$\boldsymbol{v}^T\boldsymbol{G}\boldsymbol{v} = \sum_{j=1}^{N}\sum_{i=1}^{N} v_i\phi(\boldsymbol{x}_i)^T\phi(\boldsymbol{x}_j)v_j = \sum_{j=1}^{N}\left(\sum_{i=1}^{N}\phi(\boldsymbol{x}_j)v_i\phi(\boldsymbol{x}_i)\right)^T\phi(\boldsymbol{x}_j)v_j = \left(\sum_{i=1}^{N} v_i\phi(\boldsymbol{x}_i)\right)^T\left(\sum_{j=1}^{N}\phi(\boldsymbol{x}_j)v_j\right)$$

- Because of the Positive Semi-definiteness of the inner product:

$$\phi(\boldsymbol{x}_i)^T\phi(\boldsymbol{x}_j) \geq 0$$

$$\boldsymbol{v}^T\boldsymbol{G}\boldsymbol{v} = \left(\sum_{i=1}^{N} v_i\phi(\boldsymbol{x}_i)\right)^T\left(\sum_{j=1}^{N}\phi(\boldsymbol{x}_j)v_j\right) \geq 0$$

- The Mercer's condition (PSD condition) is thus proven.

© Copyright National University of Singapore. All Rights Reserved.

What Functions Can be Kernels?

- Can we always find out a kernel K for a given mapping function ϕ?
- Yes, K always satisfies Mercer's conditions

- Given a kernel K, can we always find its corresponding ϕ function?
- No, only for K that satisfies Mercer's conditions

- What functions can be considered as candidate for kernel function K?
- The Mercer's Theorem: for $K(x_i, x_j)$ to be a valid kernel, it must satisfy the Mercer / PSD conditions.

© Copyright National University of Singapore. All Rights Reserved.

Multi-Class SVM: Classify More than 2 Classes

- All SVMs we discussed classifies between 2 classes:
  - Binary classification problem.

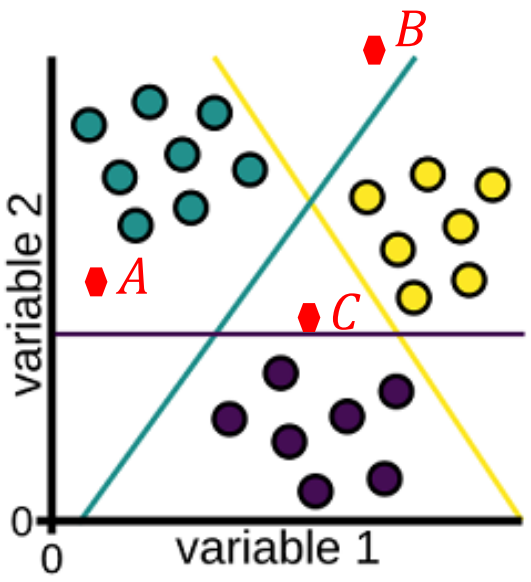
- Most tasks have much more than two classes:
  - MNIST (for digit classification): 10 classes;
  - ImageNet (for image classification): 1,000 object classes;
  - Wikipedia dataset (for text classification): 7 classes.

- How can we expand to leverage SVM for multi-class classification?
  - 1. One-vs-All approach
  - 2. One-vs-One approach

© Copyright National University of Singapore. All Rights Reserved.

Multi-Class SVM: One-vs-All

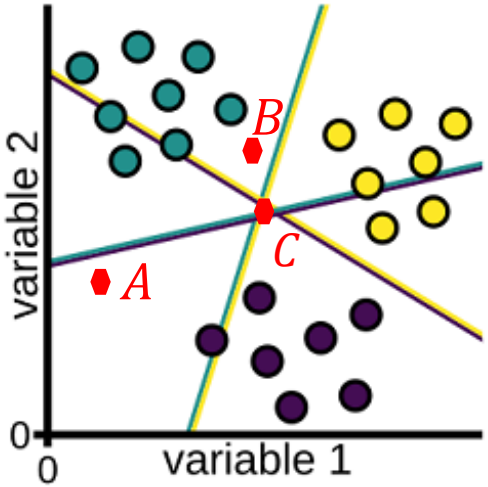
- The idea: training a binary SVM classifier for each class.
- Training phase:
  - For each class, a binary SVM classifier is trained. For a dataset with n classes, this results in n classifiers.
  - Each classifier is trained to distinguish its respective class from all other classes combined.
  - E.g., Classifier 1: Class 1 vs. the rest (Class 2 – *N*).
- Prediction phase:
  - Each classifier outputs a score or probability indicating the likelihood of the instance belonging to its respective class.
  - The class with the highest score is selected as the prediction for the instance.
- Choose the class whose classifier outputs the largest value (score).



© Copyright National University of Singapore. All Rights Reserved.

Multi-Class SVM: One-vs-One

- The idea: training a binary SVM classifier for each possible pair of class.
- Training phase:
  - For each pair of classes, a binary SVM classifier is trained.
  - Each classifier is trained using only the data points belonging to the two classes it is meant to distinguish.
  - E.g., Classifier (1, 2): Class 1 vs. Class 2.
- Prediction phase:
  - Each binary classifier provides a vote for one of the two classes it was trained on.
  - The final prediction for a given data is determined by a majority vote among all classifiers.
  - This voting mechanism ensures that the class with the most votes is selected as the final output.
- Choose the class with majority voting.



Summary

- From linear classifiers to Support Vector Machines (SVM).
- Hard-Margin SVM: classify with margins.
  - Primal form.
  - Dual form from KKT conditions.
- Soft-Margin SVM: relaxing constraints for linearly inseparable data.
  - Primal form and dual form.
  - Converting constraints to losses via the hinge loss.
- Kernel SVM: separating data in the feature space with the kernel trick.
  - The kernel trick and SVM formulation with the kernel trick.
  - Kernel functions and impact of hyperparameters.
  - The Mercer’s theorem and the Mercer’s condition.

© Copyright National University of Singapore. All Rights Reserved.